M02 — Prácticas de desarrollo de APIs REST

Autor: JAIME GALLO

Punto 2.1 — Diseño de APIs REST

Ejercicio: Añadir al punto 1.5 un documento de diseño de APIs REST en docs/, una clase de validación de URIs REST en el proyecto monolítico y un test unitario que verifique la validación. El punto 2.1 parte del contenido del punto 1.5 sin modificar ningún archivo existente: solo añade archivos nuevos. El punto 2.1 es el primer punto del módulo 2 y se crea en la carpeta modulo-2-apis-rest-aspnet-core.

Ubicación de trabajo: C:\cursos\AceriaApp\

PASO 2.1.1 — Crear la estructura del módulo 2 y del punto 2.1

- **Acción 1:** Abrir la terminal integrada de Visual Studio 2026 con Ctrl+ñ (o Ctrl+`` en teclados no españoles).
- **Acción 2:** Escribir cd C:\cursos\AceriaApp y pulsar Enter.
- **Acción 3:** Escribir mkdir modulo-2-apis-rest-aspnet-core y pulsar Enter.
- **Acción 4:** Escribir mkdir modulo-2-apis-rest-aspnet-core\2.1-diseno-apis-rest y pulsar Enter.
- **Acción 5:** Escribir xcopy modulo-1-introduccion-microservicios\1.5-buenas-practicas modulo-2-apis-rest-aspnet-core\2.1-diseno-apis-rest /E /I /H /Y y pulsar Enter.
- **Acción 6:** Escribir dir modulo-2-apis-rest-aspnet-core\2.1-diseno-apis-rest y pulsar Enter para verificar que se ha copiado el contenido.

Por qué se hace: cada punto del curso parte del contenido del punto anterior. El punto 2.1 es el primer punto del módulo 2 y parte del punto 1.5, que es el último punto del módulo 1. La copia se hace en la carpeta del módulo 2.

Errores comunes:

- Usar copy en lugar de xcopy: copy no copia carpetas de forma recursiva.
- Olvidar el flag /E: no se copian las subcarpetas vacías.
- Olvidar el flag /I: xcopy pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag /Y: xcopy pide confirmación para sobrescribir.
- Copiar desde el punto 1.4 en lugar del 1.5: se pierde el contenido del último punto del módulo 1.

Verificación: al ejecutar dir modulo-2-apis-rest-aspnet-core\2.1-diseno-apis-rest, deben aparecer AceriaApp.sln, README.md, src, tests y docs.

PASO 2.1.2 — Abrir la solución copiada en Visual Studio 2026

- **Acción 1:** Escribir cd modulo-2-apis-rest-aspnet-core\2.1-diseno-apis-rest y pulsar Enter.
- **Acción 2:** Escribir start AceriaApp.sln y pulsar Enter.
- **Acción 3:** Verificar que Visual Studio 2026 abre la solución y muestra los proyectos heredados del punto 1.5: AceriaApp.Monolito.Api, AceriaApp.Monolito.Tests, AceriaApp.Metricas.Tests, AceriaApp.Diseno.Tests, AceriaApp.Organizacion.Tests y AceriaApp.BuenasPracticas.Tests.
- **Acción 4:** Verificar que la solución compila con Ctrl+Shift+B y que la salida muestra Build succeeded..

Por qué se hace: abrir la solución copiada permite trabajar en el punto 2.1 sobre la base del punto 1.5. La compilación verifica que la copia es correcta.

Errores comunes:

- Abrir la solución del punto 1.5 en lugar de la del punto 2.1: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener la API del punto 1.5 en ejecución: el puerto queda ocupado y la compilación falla.

Verificación: Visual Studio 2026 muestra la solución AceriaApp con los seis proyectos heredados del punto 1.5 y la compilación es correcta.

PASO 2.1.3 — Verificar que la carpeta docs contiene los documentos del punto 1.5

- **Acción 1:** En el panel Solution Explorer, expandir la carpeta docs del punto 2.1.
- **Acción 2:** Verificar que contiene ADR-001 a ADR-005, analisis-ventajas-inconvenientes.md, principios-diseno.md, organizacion-solucion.md y buenas-practicas.md.
- **Acción 3:** Verificar que ninguno de los nueve documentos se ha modificado.
- **Acción 4:** Verificar que la carpeta docs no contiene ningún archivo nuevo del punto 2.1 todavía.

Por qué se hace: el punto 2.1 parte del contenido del punto 1.5, que ya incluye la carpeta docs con nueve documentos. No se modifica ningún documento existente.

Errores comunes:

- Modificar alguno de los ADR o documentos existentes: viola la regla de no modificar archivos del punto anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 1.5.
- Crear la carpeta docs si ya existe: mkdir falla.

Verificación: la carpeta docs contiene los nueve documentos del punto 1.5 sin modificaciones.

PASO 2.1.4 — Crear el documento de diseño de APIs REST en docs

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta docs.
- **Acción 2:** Seleccionar Add → New Item → Text File y nombrar el archivo diseno-apis-rest.md.
- **Acción 3:** Escribir exactamente el siguiente contenido:

```
# Diseño de APIs REST — Punto 2.1

## Restricciones de REST

1. Cliente-servidor.
2. Sin estado.
3. Cacheable.
4. Interfaz uniforme.
5. Sistema en capas.
6. Código bajo demanda (opcional).

## Recursos

Un recurso es cualquier entidad identificable. Se identifica con una URI única.

## URIs del proyecto

- `/api/v1/ordenes` → colección de órdenes.
- `/api/v1/ordenes/{id}` → orden concreta.
- `/api/v1/ordenes/{id}/lineas` → líneas de una orden.
```

```
- `/api/v1/materiales` → colección de materiales.
- `/api/v1/clientes` → colección de clientes.

## Reglas de diseño de URIs

1. Sustantivos en plural.
2. Identificador tras la colección.
3. Anidamiento de relaciones con sub-rutas.
4. Sin verbos en las URIs.
5. Versionado en la URI.

## Métodos HTTP

| Método | Semántica | Seguro | Idempotente |
|--------|-----------|--------|-------------|
| GET | Recuperar | Sí | Sí |
| POST | Crear | No | No |
| PUT | Reemplazar | No | Sí |
| PATCH | Modificar | No | No |
| DELETE | Eliminar | No | Sí |

## Códigos de estado

| Código | Significado |
|--------|-------------|
| 200 | Operación exitosa con cuerpo |
| 201 | Recurso creado |
| 204 | Operación exitosa sin cuerpo |
| 400 | Error de validación |
| 401 | Falta autenticación |
| 403 | Sin permisos |
| 404 | Recurso no encontrado |
| 409 | Conflicto de estado |
| 422 | Entidad no procesable |
| 500 | Error del servidor |

## Paginación, filtrado y ordenación

- `?page=1&pageSize=20` → paginación.
- `?filter=cliente:ACX-001` → filtrado.
- `?sort=fechaCreacion:desc` → ordenación.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Recursos | URIs con sustantivos en plural |
| Métodos | GET, POST, PUT, PATCH, DELETE |
| Códigos | 200, 201, 204, 400, 404, 409, 422, 500 |
| Representaciones | JSON |
| Paginación | `page`, `pageSize` |
| Relaciones | Sub-rutas anidadas |
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que el archivo aparece en la carpeta docs sin subrayados rojos.

Por qué se hace: el documento de diseño de APIs REST es el artefacto de docs específico del punto 2.1. No modifica los nueve documentos existentes del punto 1.5.

Errores comunes:

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.

- Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
- Guardar el archivo en la raíz del punto en lugar de docs: rompe la estructura del repositorio.

Verificación: el archivo docs\diseno-apis-rest.md existe con las nueve secciones.

PASO 2.1.5 — Crear la carpeta DisenoRest en el proyecto monolítico

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.Monolito.Api.
- **Acción 2:** Seleccionar Add → New Folder y nombrar la carpeta DisenoRest.
- **Acción 3:** Verificar que la carpeta DisenoRest aparece dentro de AceriaApp.Monolito.Api sin afectar a las carpetas Dominio, Infraestructura, Metricas, Diseno, Organizacion y BuenasPracticas.
- **Acción 4:** Verificar que no se ha modificado ningún archivo existente del punto 1.5.

Por qué se hace: la carpeta DisenoRest albergará las clases de validación de URIs REST. Se crea como carpeta nueva sin tocar las carpetas existentes.

Errores comunes:

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: viola la regla de no modificar archivos del punto anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

Verificación: la carpeta DisenoRest existe dentro de AceriaApp.Monolito.Api y las carpetas existentes permanecen intactas.

PASO 2.1.6 — Crear la clase UriRest

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta DisenoRest.
- **Acción 2:** Seleccionar Add → Class y nombrar la clase UriRest.cs.
- **Acción 3:** Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.DisenoRest;

public class UriRest
{
    public string Version { get; init; } = "v1";
    public string Recurso { get; init; } = string.Empty;
    public string? Identificador { get; init; }
    public string? SubRecurso { get; init; }

    public string Construir()
    {
        var partes = new List<string> { "api", Version, Recurso };
        if (!string.IsNullOrEmpty(Identificador))
        {
            partes.Add(Identificador);
        }
        if (!string.IsNullOrEmpty(SubRecurso))
        {
            partes.Add(SubRecurso);
        }
        return "/" + string.Join("/", partes);
    }
```

```
}
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: la clase UriRest modela la construcción de una URI REST a partir de sus partes: versión, recurso, identificador y sub-recurso. Es la base para validar que las URIs del proyecto siguen las reglas de diseño.

Errores comunes:

- Olvidar el namespace: la clase no se resuelve desde otros archivos.
- No inicializar Version con "v1": la versión no se aplica por defecto.
- Usar string en lugar de string? para las propiedades opcionales: el compilador emite advertencias.

Verificación: la clase UriRest compila sin errores y expone cuatro propiedades y un método.

PASO 2.1.7 — Añadir el using necesario en UriRest

- **Acción 1:** En el panel Solution Explorer, hacer doble clic en UriRest.cs.
- **Acción 2:** Añadir la línea using System.Collections.Generic; al principio del archivo si no está presente.
- **Acción 3:** Verificar que no hay subrayados rojos en List.
- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: el tipo List pertenece a System.Collections.Generic. En proyectos con ImplicitUsings desactivado, hay que añadir el using explícito.

Errores comunes:

- Añadir using System.Linq; en lugar de System.Collections.Generic: no resuelve List.
- Añadir el using después del namespace: el compilador no lo reconoce.
- Olvidar el using y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: UriRest.cs compila sin errores y no muestra subrayados rojos.

PASO 2.1.8 — Crear la clase ValidadorUriRest

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta DisenoRest.
- **Acción 2:** Seleccionar Add → Class y nombrar la clase ValidadorUriRest.cs.
- **Acción 3:** Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.DisenoRest;

public class ValidadorUriRest
{
    private static readonly string[] VerbosProhibidos =
    {
        "crear", "obtener", "listar", "actualizar", "eliminar",
        "get", "post", "put", "patch", "delete"
    };

    public ResultadoUri Validar(string uri)
    {
```

```
        var errores = new List<string>();

        if (!uri.StartsWith("/api/", StringComparison.Ordinal))
        {
            errores.Add("La URI no empieza por '/api/'.");
        }

        var partes = uri.Trim('/').Split('/');
        if (partes.Length >= 3 && !partes[1].StartsWith("v", StringComparison.Ordinal))
        {
            errores.Add("La URI no incluye versión después de 'api'.");
        }

        foreach (var verbo in VerbosProhibidos)
        {
            if (uri.Contains(verbo, StringComparison.OrdinalIgnoreCase))
            {
                errores.Add($"La URI contiene un verbo prohibido: '{verbo}'.");
            }
        }

        return new ResultadoUri(errores.Count == 0, errores);
    }
}

public record ResultadoUri(bool EsValida, IReadOnlyList<string> Errores);
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: la clase ValidadorUriRest comprueba que una URI sigue las reglas de diseño: empieza por /api/, incluye versión y no contiene verbos prohibidos.

Errores comunes:

- Olvidar el using System.Collections.Generic;: IReadOnlyList y List no se resuelven.
- Olvidar el using System;: StringComparison no se resuelve.
- No usar StringComparison.OrdinalIgnoreCase en la comprobación de verbos: las mayúsculas no se detectan.

Verificación: la clase ValidadorUriRest compila sin errores y expone un método y un record.

PASO 2.1.9 — Añadir los using necesarios en ValidadorUriRest

- **Acción 1:** En el panel Solution Explorer, hacer doble clic en ValidadorUriRest.cs.
- **Acción 2:** Añadir las líneas using System; y using System.Collections.Generic; al principio del archivo si no están presentes.
- **Acción 3:** Verificar que no hay subrayados rojos en StringComparison ni en IReadOnlyList.
- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: el tipo StringComparison pertenece a System. El tipo IReadOnlyList pertenece a System.Collections.Generic. En proyectos con ImplicitUsings desactivado, hay que añadir los usings explícitos.

Errores comunes:

- Añadir solo uno de los dos usings: los tipos del otro no se resuelven.
- Añadir los usings después del namespace: el compilador no los reconoce.
- Olvidar los usings y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: ValidadorUriRest.cs compila sin errores y no muestra subrayados rojos.

PASO 2.1.10 — Crear la clase CatalogoUris

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta DisenoRest.
- **Acción 2:** Seleccionar Add → Class y nombrar la clase CatalogoUris.cs.
- **Acción 3:** Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.DisenoRest;

public class CatalogoUris
{
    private readonly List<string> _uris;

    public CatalogoUris(List<string> uris)
    {
        _uris = uris;
    }

    public IEnumerable<string> UrisValidas(ValidadorUriRest validador)
    {
        return _uris.Where(u => validador.Validar(u).EsValida);
    }

    public IEnumerable<string> UrisInvalidas(ValidadorUriRest validador)
    {
        return _uris.Where(u => !validador.Validar(u).EsValida);
    }
}
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: la clase CatalogoUris agrupa las URIs del proyecto y permite filtrarlas por validez según el validador.

Errores comunes:

- No inicializar _uris en el constructor: la clase lanza NullReferenceException.
- Olvidar using System.Linq;: Where no se resuelve.
- Devolver List en lugar de IEnumerable: expone la implementación interna.

Verificación: la clase CatalogoUris compila sin errores y expone un constructor y dos métodos.

PASO 2.1.11 — Añadir el using necesario en CatalogoUris

- **Acción 1:** En el panel Solution Explorer, hacer doble clic en CatalogoUris.cs.
- **Acción 2:** Añadir la línea using System.Linq; al principio del archivo si no está presente.
- **Acción 3:** Verificar que no hay subrayados rojos en Where.
- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: el método Where pertenece a System.Linq. En proyectos con ImplicitUsings desactivado, hay que añadir el using explícito.

Errores comunes:

- Añadir using System.Collections.Generic; en lugar de System.Linq: no resuelve Where.

- Añadir el using después del namespace: el compilador no lo reconoce.
- Olvidar el using y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: CatalogoUris.cs compila sin errores y no muestra subrayados rojos.

PASO 2.1.12 — Crear el proyecto de tests del punto 2.1

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta tests.
- **Acción 2:** Seleccionar Add → New Project.
- **Acción 3:** Seleccionar la plantilla xUnit Test Project, nombrar el proyecto AceriaApp.DisenoRest.Tests y pulsar Create.
- **Acción 4:** Verificar que el proyecto aparece en la solución y que se ha creado la carpeta tests\AceriaApp.DisenoRest.Tests.

Por qué se hace: el punto 2.1 añade un proyecto de tests específico para las clases de diseño REST. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

Errores comunes:

- Crear el proyecto dentro de src en lugar de tests: rompe la estructura.
- Modificar los proyectos de tests del punto 1.5: viola la regla de no modificar archivos del punto anterior.
- Usar dotnet new mstest: el curso usa xUnit.

Verificación: el proyecto AceriaApp.DisenoRest.Tests aparece en la solución y su carpeta existe en tests\AceriaApp.DisenoRest.Tests.

PASO 2.1.13 — Añadir la referencia del proyecto de tests a la API monolítica

- **Acción 1:** En la terminal, escribir dotnet add tests\AceriaApp.DisenoRest.Tests\AceriaApp.DisenoRest.Tests.csproj reference src\AceriaApp.Monolito.Api\AceriaApp.Monolito.Api.csproj y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra Reference added successfully.
- **Acción 3:** Verificar que el archivo AceriaApp.DisenoRest.Tests.csproj contiene la referencia.

Por qué se hace: la referencia permite que el proyecto de tests acceda a las clases de DisenoRest del proyecto monolítico.

Errores comunes:

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo .csproj: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

Verificación: el archivo AceriaApp.DisenoRest.Tests.csproj contiene un ProjectReference al proyecto monolítico.

PASO 2.1.14 — Añadir el paquete FluentAssertions al proyecto de tests

- **Acción 1:** Escribir dotnet add tests\AceriaApp.DisenoRest.Tests\AceriaApp.DisenoRest.Tests.csproj package FluentAssertions y pulsar Enter.
- **Acción 2:** Escribir dotnet list tests\AceriaApp.DisenoRest.Tests\AceriaApp.DisenoRest.Tests.csproj package y pulsar Enter.
- **Acción 3:** Verificar que la salida muestra FluentAssertions con su versión.

Por qué se hace: FluentAssertions proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

Errores comunes:

- Instalar una versión preliminar: el curso usa la última estable.
- Olvidar el paquete: Should() no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

Verificación: la salida de dotnet list package muestra FluentAssertions con una versión estable.

PASO 2.1.15 — Crear el archivo ValidadorUriRestTests.cs

- **Acción 1:** En el panel Solution Explorer, expandir tests → AceriaApp.DisenoRest.Tests.
- **Acción 2:** Hacer doble clic en UnitTest1.cs y renombrarlo a ValidadorUriRestTests.cs.
- **Acción 3:** Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.DisenoRest;
using FluentAssertions;
using Xunit;

namespace AceriaApp.DisenoRest.Tests;

public class ValidadorUriRestTests
{
    private readonly ValidadorUriRest _validador = new();

    [Fact]
    public void Validar_UriCorrecta_DevuelveValida()
    {
        var resultado = _validador.Validar("/api/v1/ordenes");

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinPrefijoApi_DevuelveError()
    {
        var resultado = _validador.Validar("/ordenes");

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no empieza por"));
    }

    [Fact]
    public void Validar_SinVersion_DevuelveError()
    {
        var resultado = _validador.Validar("/api/ordenes");

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no incluye versión"));
    }

    [Fact]
    public void Validar_ConVerbo_DevuelveError()
    {
        var resultado = _validador.Validar("/api/v1/crearOrden");

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("verbo prohibido"));
```

```
    }
}
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: los tests verifican que el validador acepta URIs correctas y rechaza URIs sin prefijo, sin versión y con verbos prohibidos.

Errores comunes:

- Olvidar el using AceriaApp.Monolito.Api.DisenoRest;: las clases no se resuelven.
- Usar Assert.True en lugar de Should().BeTrue(): el curso usa FluentAssertions.
- No cubrir el caso de URI correcta: el test solo verifica los errores.

Verificación: el archivo ValidadorUriRestTests.cs compila y contiene cuatro métodos de test.

PASO 2.1.16 — Crear el archivo UriRestTests.cs

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.DisenoRest.Tests.
- **Acción 2:** Seleccionar Add → Class y nombrar la clase UriRestTests.cs.
- **Acción 3:** Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.DisenoRest;
using FluentAssertions;
using Xunit;

namespace AceriaApp.DisenoRest.Tests;

public class UriRestTests
{
    [Fact]
    public void Construir_SinIdentificadorNiSubRecurso_DevuelveUriBase()
    {
        var uri = new UriRest { Recurso = "ordenes" };

        uri.Construir().Should().Be("/api/v1/ordenes");
    }

    [Fact]
    public void Construir_ConIdentificador_DevuelveUriConId()
    {
        var uri = new UriRest { Recurso = "ordenes", Identificador = "123" };

        uri.Construir().Should().Be("/api/v1/ordenes/123");
    }

    [Fact]
    public void Construir_ConSubRecurso_DevuelveUriAnidada()
    {
        var uri = new UriRest
        {
            Recurso = "ordenes",
            Identificador = "123",
            SubRecurso = "lineas"
```

```
        };

        uri.Construir().Should().Be("/api/v1/ordenes/123/lineas");
    }
}
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: los tests verifican que la clase UriRest construye correctamente las URIs en los tres casos: sin identificador, con identificador y con sub-recurso.

Errores comunes:

- Olvidar el using AceriaApp.Monolito.Api.DisenoRest;: las clases no se resuelven.
- Usar Assert.Equal en lugar de Should().Be(): el curso usa FluentAssertions.
- No cubrir el caso de sub-recurso: la anidación no se verifica.

Verificación: el archivo UriRestTests.cs compila y contiene tres métodos de test.

PASO 2.1.17 — Crear el archivo CatalogoUrisTests.cs

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.DisenoRest.Tests.
- **Acción 2:** Seleccionar Add → Class y nombrar la clase CatalogoUrisTests.cs.
- **Acción 3:** Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.DisenoRest;
using FluentAssertions;
using Xunit;

namespace AceriaApp.DisenoRest.Tests;

public class CatalogoUrisTests
{
    [Fact]
    public void UrisValidas_FiltraCorrectamente()
    {
        var uris = new List<string>
        {
            "/api/v1/ordenes",
            "/api/v1/ordenes/123",
            "/ordenes",
            "/api/v1/crearOrden"
        };
        var catalogo = new CatalogoUris(uris);
        var validador = new ValidadorUriRest();

        var validas = catalogo.UrisValidas(validador).ToList();
        var invalidas = catalogo.UrisInvalidas(validador).ToList();

        validas.Should().HaveCount(2);
        invalidas.Should().HaveCount(2);
    }
}
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.

- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: el test verifica que el catálogo filtra correctamente las URIs válidas y las inválidas según el validador.

Errores comunes:

- Olvidar using System.Collections.Generic;: List no se resuelve.
- Olvidar using System.Linq;: ToList() no se resuelve.
- No comprobar los recuentos: el test podría pasar por casualidad.

Verificación: el archivo CatalogoUrisTests.cs compila y contiene un método de test.

PASO 2.1.18 — Compilar la solución completa

- **Acción 1:** Escribir dotnet build AceriaApp.sln y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra Build succeeded. y 0 Warning(s), 0 Error(s).
- **Acción 3:** Comprobar que el comando termina sin errores y que el resultado coincide con la verificación del ejercicio.

Por qué se hace: compilar la solución completa verifica que todos los proyectos, incluidos los nuevos, se resuelven correctamente.

Errores comunes:

- Tener la API en ejecución: el archivo .dll está bloqueado y la compilación falla.
- No restaurar paquetes: dotnet build restaura por defecto, pero si hay un error de red, hay que ejecutar dotnet restore primero.
- Usar una versión distinta del SDK: dotnet --version debe devolver 8.x.x.

Verificación: la salida debe mostrar Build succeeded. con 0 Error(s).

PASO 2.1.19 — Ejecutar todos los tests del punto 2.1

- **Acción 1:** Escribir dotnet test tests\AceriaApp.DisenoRest.Tests\AceriaApp.DisenoRest.Tests.csproj y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra Passed! - Failed: 0, Passed: 8, Skipped: 0.
- **Acción 3:** Comprobar que el comando termina sin errores y que el resultado coincide con la verificación del ejercicio.

Por qué se hace: ejecutar los tests del punto 2.1 verifica que las clases de diseño REST funcionan correctamente.

Errores comunes:

- Ejecutar dotnet test sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: dotnet test compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 8, Skipped: 0, Total: 8.

PASO 2.1.20 — Ejecutar todos los tests de la solución

- **Acción 1:** Escribir dotnet test AceriaApp.sln --no-build y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra Passed! - Failed: 0, Passed: 31, Skipped: 0.
- **Acción 3:** Comprobar que el comando termina sin errores y que el resultado coincide con la verificación del ejercicio.

Por qué se hace: ejecutar todos los tests de la solución verifica que los tests de los puntos 1.1 a 1.5 y del punto 2.1 pasan conjuntamente.

Errores comunes:

- No haber compilado antes: --no-build falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 31, Skipped: 0, Total: 31.

PASO 2.1.21 — Crear el README del punto 2.1

- **Acción 1:** En la carpeta del punto 2.1-diseno-apis-rest, hacer clic derecho y seleccionar Add → New Item → Text File.
- **Acción 2:** Nombrar el archivo README.md.
- **Acción 3:** Escribir exactamente el siguiente contenido:

```
# Punto 2.1 — Diseño de APIs REST

## Objetivo
Comprender los principios del estilo arquitectónico REST y añadir al proyecto las clases de
validación de URIs REST.

## Estado del proyecto
- Solución `AceriaApp.sln` con siete proyectos: `AceriaApp.Monolito.Api`,
`AceriaApp.Monolito.Tests`, `AceriaApp.Metricas.Tests`, `AceriaApp.Diseno.Tests`,
`AceriaApp.Organizacion.Tests`, `AceriaApp.BuenasPracticas.Tests`, `AceriaApp.DisenoRest.Tests` y
los proyectos heredados.
- Documento `docs/diseno-apis-rest.md` con las restricciones de REST, las reglas de URIs, los
métodos HTTP, los códigos de estado y la paginación.
- Clases `UriRest`, `ValidadorUriRest` y `CatalogoUris` en `src/AceriaApp.Monolito.Api/DisenoRest/
`.
- Ocho tests nuevos en `tests/AceriaApp.DisenoRest.Tests`.
- Los tests de los puntos 1.1 a 1.5 y del punto 2.1 suman 31 pruebas unitarias.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Ejecutar `dotnet ef database update` en `src\AceriaApp.Monolito.Api`.
4. Ejecutar `dotnet run` en `src\AceriaApp.Monolito.Api`.
5. Navegar a `https://localhost:7xxx/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/diseno-apis-rest.md`.
- Carpeta `DisenoRest` en el proyecto monolítico con tres clases nuevas.
- Proyecto `tests/AceriaApp.DisenoRest.Tests` con ocho tests nuevos.
- No se ha modificado ningún archivo del punto 1.5.

## Documentación
- [ADR-001](docs/ADR-001-monolito-vs-microservicios.md) (heredado del punto 1.1).
- [ADR-002](docs/ADR-002-metricas-acoplamiento.md) (heredado del punto 1.2).
- [ADR-003](docs/ADR-003-principios-diseno.md) (heredado del punto 1.3).
- [ADR-004](docs/ADR-004-organizacion-solucion.md) (heredado del punto 1.4).
- [ADR-005](docs/ADR-005-buenas-practicas.md) (heredado del punto 1.5).
- [Análisis de ventajas e inconvenientes](docs/analisis-ventajas-inconvenientes.md) (heredado del
```

```
punto 1.2).
- [Principios de diseño](docs/principios-diseno.md) (heredado del punto 1.3).
- [Organización de la solución](docs/organizacion-solucion.md) (heredado del punto 1.4).
- [Buenas prácticas](docs/buenas-practicas.md) (heredado del punto 1.5).
- [Diseño de APIs REST](docs/diseno-apis-rest.md).
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que el README del punto 1.5 no se ha modificado.

Por qué se hace: el README del punto 2.1 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 1.5.

Errores comunes:

- Modificar el README del punto 1.5: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección Qué se ha añadido respecto al punto anterior: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 1.5: rompe la separación.

Verificación: el archivo README.md existe en la carpeta del punto 2.1 con las cinco secciones.

PASO 2.1.22 — Crear el ADR-006 del punto 2.1

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta docs.
- **Acción 2:** Seleccionar Add → New Item → Text File y nombrar el archivo ADR-006-diseno-apis-rest.md.
- **Acción 3:** Escribir exactamente el siguiente contenido:

```
# ADR-006: Diseño de APIs REST

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha cerrado el módulo 1 con un monolito modular, métricas de acoplamiento,
principios de diseño, organización y buenas prácticas. Es necesario adoptar un estilo de diseño
para las APIs REST que se construirán en el módulo 2.

## Decisión
Se adopta el estilo arquitectónico REST con las seis restricciones, las cinco reglas de diseño de
URIs, la semántica de los métodos HTTP, los códigos de estado adecuados a cada situación y la
paginación con parámetros de consulta. Se implementa un validador automático de URIs REST.

## Alternativas consideradas
- Estilo RPC con URIs de acciones: descartado por falta de uniformidad.
- Estilo GraphQL: descartado por el alcance del curso.

## Consecuencias
- Positivas: uniformidad, previsibilidad, documentación automática, facilidad de integración.
- Negativas: el validador puede requerir ajuste de reglas.
- Mitigación: revisar las reglas al final del módulo 2.
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que los ADR-001 a ADR-005 no se han modificado.

Por qué se hace: el ADR-006 documenta la decisión de adoptar el estilo REST y de validar automáticamente las URIs. Es un artefacto nuevo del punto 2.1 que no modifica los ADR existentes.

Errores comunes:

- Modificar los ADR anteriores: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección Alternativas consideradas: el ADR pierde valor.
- Nombrar el archivo sin el prefijo ADR-006: rompe la convención.

Verificación: el archivo docs\ADR-006-diseno-apis-rest.md existe con las cinco secciones.

PASO 2.1.23 — Verificar que no se ha modificado ningún archivo del punto 1.5

- **Acción 1:** En la terminal, escribir cd C:\cursos\AceriaApp y pulsar Enter.
- **Acción 2:** Escribir fc /L modulo-1-introduccion-microservicios\1.5-buenas-practicas\docs\ADR-005-buenas-practicas.md modulo-2-apis-rest-aspnet-core\2.1-diseno-apis-rest\docs\ADR-005-buenas-practicas.md y pulsar Enter.
- **Acción 3:** Verificar que la salida muestra FC: no differences encountered.
- **Acción 4:** Escribir fc /L modulo-1-introduccion-microservicios\1.5-buenas-practicas\docs\buenas-practicas.md modulo-2-apis-rest-aspnet-core\2.1-diseno-apis-rest\docs\buenas-practicas.md y pulsar Enter.
- **Acción 5:** Verificar que la salida muestra FC: no differences encountered.

Por qué se hace: la comparación con fc verifica que los documentos del punto 1.5 no se han modificado.

Errores comunes:

- Usar comp en lugar de fc: comp compara binarios y no muestra diferencias de texto.
- Olvidar el flag /L: fc compara en modo ASCII y muestra líneas largas sin ajuste.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

Verificación: los dos documentos del punto 1.5 no muestran diferencias.

PASO 2.1.24 — Hacer el commit del punto 2.1 y verificar el estado final

- **Acción 1:** Escribir git add . y pulsar Enter.
- **Acción 2:** Escribir git commit -m "Punto 2.1: diseño de APIs REST y validación de URIs" y pulsar Enter.
- **Acción 3:** Escribir git log --oneline y pulsar Enter para verificar los seis commits.
- **Acción 4:** Escribir git status y pulsar Enter para verificar que el árbol está limpio.
- **Acción 5:** Escribir tree /F modulo-2-apis-rest-aspnet-core\2.1-diseno-apis-rest y pulsar Enter para verificar la estructura final del punto.

Por qué se hace: el commit del punto 2.1 registra el estado acumulado del proyecto. Los seis commits consecutivos permiten comparar el estado del proyecto entre puntos consecutivos y entre módulos completos.

Errores comunes:

- No tener configurado user.name y user.email: git commit falla.
- Añadir la carpeta bin/ y obj/ al commit: el .gitignore lo evita.
- Hacer commit antes de git add .: no hay cambios en el índice.

Verificación: git log --oneline muestra seis commits: del punto 1.1 al 1.5 y del punto 2.1. git status muestra nothing to commit, working tree clean.

Errores comunes del ejercicio completo

Error observado	Causa	Solución guiada
xcopy no copia subcarpetas	Falta el flag /E	Ejecutar xcopy origen destino /E /I /H /Y
StringComparison no se resuelve	Falta el using System;	Añadir el using al principio del archivo
IReadOnlyList no se resuelve	Falta el using System.Collections.Generic;	Añadir el using al principio del archivo
Where no se resuelve	Falta el using System.Linq;	Añadir el using al principio del archivo
NullReferenceException en CatalogoUris	No se inicializa _uris	Inicializar en el constructor
El validador no detecta URIs sin prefijo	Falta la comprobación con StartsWith	Revisar la primera comprobación de Validar
El validador no detecta verbos en mayúsculas	Falta StringComparison.OrdinalIgnoreCase	Añadir el parámetro a Contains
Should() no se resuelve	Falta el paquete FluentAssertions	Ejecutar dotnet add package FluentAssertions
El proyecto de tests no compila	Falta la referencia al proyecto monolítico	Ejecutar dotnet add reference
La compilación falla por el .dll bloqueado	La API está en ejecución	Detener la API con Ctrl+C

Reto resuelto paso a paso

Reto: añadir un método Resumen a CatalogoUris que devuelva el número de URIs válidas, el número de inválidas y el porcentaje de validez, y un test que verifique el cálculo cuando hay tres válidas y dos inválidas.

Paso 1: En CatalogoUris.cs, añadir el siguiente record al final del archivo:

```
public record ResumenUris(int Validas, int Invalidas, double PorcentajeValidez);
```

Paso 2: Explicar línea por línea:

- **Línea 1:** public record ResumenUris(→ declara un contrato inmutable.
- **Línea 2:** int Validas, → número de URIs válidas.
- **Línea 3:** int Invalidas, → número de URIs inválidas.
- **Línea 4:** double PorcentajeValidez); → porcentaje de validez.

Paso 3: En CatalogoUris.cs, añadir el siguiente método dentro de la clase:

```
public ResumenUris Resumen(ValidadorUriRest validador)
{
    var validas = _uris.Count(u => validador.Validar(u).EsValida);
    var invalidas = _uris.Count - validas;
    var total = _uris.Count;
    var porcentaje = total == 0 ? 0 : (double)validas / total * 100;
    return new ResumenUris(validas, invalidas, porcentaje);
```

```
}
```

Paso 4: Explicar línea por línea:

- **Línea 1:** public ResumenUris Resumen(ValidadorUriRest validador) → declara el método.
- **Línea 2:** { → abre el cuerpo.
- **Línea 3:** var validas = _uris.Count(u => validador.Validar(u).EsValida); → cuenta las válidas.
- **Línea 4:** var invalidas = _uris.Count - validas; → calcula las inválidas por diferencia.
- **Línea 5:** var total = _uris.Count; → obtiene el total.
- **Línea 6:** var porcentaje = total == 0 ? 0 : (double)validas / total * 100; → calcula el porcentaje con protección contra división por cero.
- **Línea 7:** return new ResumenUris(validas, invalidas, porcentaje); → devuelve el resumen.
- **Línea 8:** } → cierra el cuerpo.

Paso 5: Guardar el archivo con Ctrl+S.

Paso 6: En CatalogoUrisTests.cs, añadir el siguiente test:

```
[Fact]
public void Resumen_ConTresValidasYDosInvalidas_CalculaPorcentaje()
{
    var uris = new List<string>
    {
        "/api/v1/ordenes",
        "/api/v1/ordenes/123",
        "/api/v1/materiales",
        "/ordenes",
        "/api/v1/crearOrden"
    };
    var catalogo = new CatalogoUris(uris);
    var validador = new ValidadorUriRest();

    var resumen = catalogo.Resumen(validador);

    resumen.Validas.Should().Be(3);
    resumen.Invalidas.Should().Be(2);
    resumen.PorcentajeValidez.Should().Be(60.0);
}
```

Paso 7: Explicar línea por línea:

- **Línea 1:** [Fact] → marca el método como test de xUnit.
- **Línea 2:** public void Resumen_ConTresValidasYDosInvalidas_CalculaPorcentaje() → declara el test.
- **Línea 3:** { → abre el cuerpo.
- **Línea 4:** var uris = new List → crea la lista.
- **Línea 5:** { → abre el inicializador.
- **Línea 6:** "/api/v1/ordenes", → primera URI válida.
- **Línea 7:** "/api/v1/ordenes/123", → segunda URI válida.
- **Línea 8:** "/api/v1/materiales", → tercera URI válida.
- **Línea 9:** "/ordenes", → primera URI inválida por falta de prefijo.
- **Línea 10:** "/api/v1/crearOrden" → segunda URI inválida por contener verbo.
- **Línea 11:** }; → cierra el inicializador.
- **Línea 12:** var catalogo = new CatalogoUris(uris); → instancia el catálogo.
- **Línea 13:** var validador = new ValidadorUriRest(); → instancia el validador.

- **Línea 14:** var resumen = catalogo.Resumen(validador); → invoca el método.
- **Línea 15:** resumen.Validas.Should().Be(3); → verifica las válidas.
- **Línea 16:** resumen.Invalidas.Should().Be(2); → verifica las inválidas.
- **Línea 17:** resumen.PorcentajeValidez.Should().Be(60.0); → verifica el porcentaje.
- **Línea 18:** } → cierra el cuerpo.

Paso 8: Ejecutar dotnet test tests\AceriaApp.DisenoRest.Tests\AceriaApp.DisenoRest.Tests.csproj y verificar que pasan los nueve tests.

Paso 9: Verificar que el método Resumen no modifica ningún archivo del punto 1.5.

Paso 10: Hacer commit con git commit -m "Reto 2.1: método Resumen en CatalogoUris con cálculo de porcentaje".

Verificación: la salida de dotnet test muestra Passed: 9.

Resultado esperado

Al finalizar este punto, el alumno debe tener:

- La carpeta modulo-2-apis-rest-aspnet-core/2.1-diseno-apis-rest/ con la solución AceriaApp.sln que contiene ocho proyectos.
- El documento docs/diseno-apis-rest.md con las restricciones, reglas, métodos, códigos y paginación.
- El documento docs/ADR-006-diseno-apis-rest.md con la decisión sobre el estilo REST.
- Las clases UriRest, ValidadorUriRest y CatalogoUris en src/AceriaApp.Monolito.Api/DisenoRest/.
- El proyecto tests/AceriaApp.DisenoRest.Tests con ocho tests (nueve tras el reto).
- Los tests de los puntos 1.1 a 1.5 y del punto 2.1 pasando conjuntamente con dotnet test AceriaApp.sln, total 31 pruebas (32 tras el reto).
- Los ADR-001 a ADR-005 del punto 1.5 sin modificar.
- Seis commits en Git: del punto 1.1 al 1.5 y del punto 2.1.

Conclusión y enlace al siguiente punto

El punto 2.1 ha establecido los fundamentos del diseño de APIs REST: las seis restricciones del estilo, la identificación de recursos con URIs, la negociación de representaciones, la semántica de los métodos HTTP, los códigos de estado, la paginación y el anidamiento de relaciones. El proyecto AceriaApp dispone ahora de un documento de diseño de APIs REST, un validador automático de URIs y ocho tests nuevos. Esta base permite avanzar hacia el punto 2.2, donde se estudiarán con detalle los recursos, los verbos HTTP y los códigos de respuesta, y se aplicarán al diseño de los endpoints del proyecto.

Fin del punto 2.1.

Punto 2.2 — Recursos, verbos HTTP y códigos de respuesta

Ejercicio: Añadir al punto 2.1 un documento de catálogo de recursos en docs/, una clase de validación de combinaciones verbo-código en el proyecto monolítico y un test unitario que verifique la validación. El punto 2.2 parte del contenido del punto 2.1 sin modificar ningún archivo existente: solo añade archivos nuevos.

Ubicación de trabajo: C:\cursos\AceriaApp\modulo-2-apis-rest-aspnet-core\

PASO 2.2.1 — Crear la carpeta del punto 2.2 a partir del punto 2.1

- **Acción 1:** Abrir la terminal integrada de Visual Studio 2026 con Ctrl+ñ (o Ctrl+`` en teclados no españoles).
- **Acción 2:** Escribir cd C:\cursos\AceriaApp\modulo-2-apis-rest-aspnet-core y pulsar Enter.
- **Acción 3:** Escribir mkdir 2.2-recursos-verbos-codigos y pulsar Enter.
- **Acción 4:** Escribir xcopy 2.1-diseno-apis-rest 2.2-recursos-verbos-codigos /E /I /H /Y y pulsar Enter.
- **Acción 5:** Escribir dir 2.2-recursos-verbos-codigos y pulsar Enter para verificar que se ha copiado el contenido.

Por qué se hace: cada punto del curso parte del contenido del punto anterior. El punto 2.2 se construye sobre el punto 2.1 copiando su contenido y añadiendo archivos nuevos sin modificar los existentes.

Errores comunes:

- Usar copy en lugar de xcopy: copy no copia carpetas de forma recursiva.
- Olvidar el flag /E: no se copian las subcarpetas vacías.
- Olvidar el flag /I: xcopy pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag /Y: xcopy pide confirmación para sobrescribir.

Verificación: al ejecutar dir 2.2-recursos-verbos-codigos, deben aparecer AceriaApp.sln, README.md, src, tests y docs.

PASO 2.2.2 — Abrir la solución copiada en Visual Studio 2026

- **Acción 1:** Escribir cd 2.2-recursos-verbos-codigos y pulsar Enter.
- **Acción 2:** Escribir start AceriaApp.sln y pulsar Enter.
- **Acción 3:** Verificar que Visual Studio 2026 abre la solución y muestra los proyectos heredados del punto 2.1: AceriaApp.Monolito.Api, AceriaApp.Monolito.Tests, AceriaApp.Metricas.Tests, AceriaApp.Diseno.Tests, AceriaApp.Organizacion.Tests, AceriaApp.BuenasPracticas.Tests y AceriaApp.DisenoRest.Tests.
- **Acción 4:** Verificar que la solución compila con Ctrl+Shift+B y que la salida muestra Build succeeded..

Por qué se hace: abrir la solución copiada permite trabajar en el punto 2.2 sobre la base del punto 2.1. La compilación verifica que la copia es correcta.

Errores comunes:

- Abrir la solución del punto 2.1 en lugar de la del punto 2.2: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener la API del punto 2.1 en ejecución: el puerto queda ocupado y la compilación falla.

Verificación: Visual Studio 2026 muestra la solución AceriaApp con los siete proyectos heredados del punto 2.1 y la compilación es correcta.

PASO 2.2.3 — Verificar que la carpeta docs contiene los documentos del punto 2.1

- **Acción 1:** En el panel Solution Explorer, expandir la carpeta docs del punto 2.2.
- **Acción 2:** Verificar que contiene ADR-001 a ADR-006, analisis-ventajas-inconvenientes.md, principios-diseno.md, organizacion-solucion.md, buenas-practicas.md y diseno-apis-rest.md.
- **Acción 3:** Verificar que ninguno de los once documentos se ha modificado.
- **Acción 4:** Verificar que la carpeta docs no contiene ningún archivo nuevo del punto 2.2 todavía.

Por qué se hace: el punto 2.2 parte del contenido del punto 2.1, que ya incluye la carpeta docs con once documentos. No se modifica ningún documento existente.

Errores comunes:

- Modificar alguno de los ADR o documentos existentes: viola la regla de no modificar archivos del punto anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 2.1.
- Crear la carpeta docs si ya existe: mkdir falla.

Verificación: la carpeta docs contiene los once documentos del punto 2.1 sin modificaciones.

PASO 2.2.4 — Crear el documento de catálogo de recursos en docs

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta docs.
- **Acción 2:** Seleccionar Add → New Item → Text File y nombrar el archivo catalogo-recursos.md.
- **Acción 3:** Escribir exactamente el siguiente contenido:

```
# Catálogo de recursos — Punto 2.2

## Recursos del dominio

- Orden.
- LineaOrden.
- Material.
- Cliente.
- Movimiento.

## Catálogo de combinaciones verbo-URI

| Recurso | URI base | Verbos permitidos |
|---------|----------|-------------------|
| Orden | /api/v1/ordenes | GET, POST |
| Orden | /api/v1/ordenes/{id} | GET, PUT, PATCH, DELETE |
| LineaOrden | /api/v1/ordenes/{id}/lineas | GET, POST |
| LineaOrden | /api/v1/ordenes/{id}/lineas/{lid} | GET, PUT, DELETE |
| Material | /api/v1/materiales | GET, POST |
| Material | /api/v1/materiales/{id} | GET, PUT, PATCH, DELETE |
| Cliente | /api/v1/clientes | GET, POST |
| Cliente | /api/v1/clientes/{id} | GET, PUT, DELETE |

## Códigos de respuesta por verbo

| Verbo | Códigos esperados |
|-------|-------------------|
| GET | 200, 404 |
| POST | 201, 400, 409, 422 |
| PUT | 200, 204, 400, 404, 409, 422 |
| PATCH | 200, 204, 400, 404, 409, 422 |
| DELETE | 204, 404 |

## Formato de error

Problem details según RFC 7807.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Recursos | Orden, LineaOrden, Material, Cliente, Movimiento |
| Verbos | GET, POST, PUT, PATCH, DELETE |
```

```
| Códigos | 200, 201, 204, 400, 404, 409, 422, 500 |
| Formato de error | Problem details |
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que el archivo aparece en la carpeta docs sin subrayados rojos.

Por qué se hace: el documento de catálogo de recursos es el artefacto de docs específico del punto 2.2. No modifica los once documentos existentes del punto 2.1.

Errores comunes:

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
- Guardar el archivo en la raíz del punto en lugar de docs: rompe la estructura del repositorio.

Verificación: el archivo docs\catalogo-recursos.md existe con las seis secciones.

PASO 2.2.5 — Crear la carpeta CatalogoRecursos en el proyecto monolítico

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.Monolito.Api.
- **Acción 2:** Seleccionar Add → New Folder y nombrar la carpeta CatalogoRecursos.
- **Acción 3:** Verificar que la carpeta CatalogoRecursos aparece dentro de AceriaApp.Monolito.Api sin afectar a las carpetas Dominio, Infraestructura, Metricas, Diseno, Organizacion, BuenasPracticas y DisenoRest.
- **Acción 4:** Verificar que no se ha modificado ningún archivo existente del punto 2.1.

Por qué se hace: la carpeta CatalogoRecursos albergará las clases de catálogo y validación de combinaciones verbo-código. Se crea como carpeta nueva sin tocar las carpetas existentes.

Errores comunes:

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: viola la regla de no modificar archivos del punto anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

Verificación: la carpeta CatalogoRecursos existe dentro de AceriaApp.Monolito.Api y las carpetas existentes permanecen intactas.

PASO 2.2.6 — Crear la clase CombinacionVerboCodigo

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta CatalogoRecursos.
- **Acción 2:** Seleccionar Add → Class y nombrar la clase CombinacionVerboCodigo.cs.
- **Acción 3:** Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.CatalogoRecursos;

public class CombinacionVerboCodigo
{
    public string Verbo { get; init; } = string.Empty;
    public int Codigo { get; init; }
}
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.

- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: la clase CombinacionVerboCodigo modela la pareja de verbo y código de respuesta que el validador comprobará. Es la unidad mínima de validación.

Errores comunes:

- Olvidar el namespace: la clase no se resuelve desde otros archivos.
- No inicializar Verbo con string.Empty: el compilador emite advertencia de nulabilidad.
- Usar set en lugar de init: permite modificar la instancia después de crearla.

Verificación: la clase CombinacionVerboCodigo compila sin errores y expone dos propiedades.

PASO 2.2.7 — Crear la clase ValidadorCombinacionVerboCodigo

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta CatalogoRecursos.
- **Acción 2:** Seleccionar Add → Class y nombrar la clase ValidadorCombinacionVerboCodigo.cs.
- **Acción 3:** Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.CatalogoRecursos;

public class ValidadorCombinacionVerboCodigo
{
    private static readonly Dictionary<string, int[]> CodigosPorVerbo = new()
    {
        { "GET", new[] { 200, 404 } },
        { "POST", new[] { 201, 400, 409, 422 } },
        { "PUT", new[] { 200, 204, 400, 404, 409, 422 } },
        { "PATCH", new[] { 200, 204, 400, 404, 409, 422 } },
        { "DELETE", new[] { 204, 404 } }
    };

    public ResultadoCombinacion Validar(CombinacionVerboCodigo combinacion)
    {
        var errores = new List<string>();

        if (!CodigosPorVerbo.ContainsKey(combinacion.Verbo))
        {
            errores.Add($"Verbo no permitido: '{combinacion.Verbo}'.");
            return new ResultadoCombinacion(false, errores);
        }

        if (!CodigosPorVerbo[combinacion.Verbo].Contains(combinacion.Codigo))
        {
            errores.Add($"Código '{combinacion.Codigo}' no permitido para el verbo
'{combinacion.Verbo}'.");
        }

        return new ResultadoCombinacion(errores.Count == 0, errores);
    }
}

public record ResultadoCombinacion(bool EsValida, IReadOnlyList<string> Errores);
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: la clase ValidadorCombinacionVerboCodigo comprueba que un verbo está permitido y que el código de respuesta es coherente con el verbo. La tabla de códigos por verbo está definida como un diccionario estático.

Errores comunes:

- Olvidar el using System.Collections.Generic;: Dictionary y List no se resuelven.
- Olvidar el using System.Linq;: Contains no se resuelve.
- No comprobar si el verbo existe antes de acceder al diccionario: lanza KeyNotFoundException.

Verificación: la clase ValidadorCombinacionVerboCodigo compila sin errores y expone un método y un record.

PASO 2.2.8 — Añadir los using necesarios en ValidadorCombinacionVerboCodigo

- **Acción 1:** En el panel Solution Explorer, hacer doble clic en ValidadorCombinacionVerboCodigo.cs.
- **Acción 2:** Añadir las líneas using System.Collections.Generic; y using System.Linq; al principio del archivo si no están presentes.
- **Acción 3:** Verificar que no hay subrayados rojos en Dictionary, List ni en Contains.
- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: los tipos Dictionary y List pertenecen a System.Collections.Generic. El método Contains pertenece a System.Linq. En proyectos con ImplicitUsings desactivado, hay que añadir los usings explícitos.

Errores comunes:

- Añadir solo System.Collections.Generic y olvidar System.Linq: Contains no se resuelve.
- Añadir los usings después del namespace: el compilador no los reconoce.
- Olvidar los usings y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: ValidadorCombinacionVerboCodigo.cs compila sin errores y no muestra subrayados rojos.

PASO 2.2.9 — Crear la clase CatalogoCombinaciones

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta CatalogoRecursos.
- **Acción 2:** Seleccionar Add → Class y nombrar la clase CatalogoCombinaciones.cs.
- **Acción 3:** Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.CatalogoRecursos;

public class CatalogoCombinaciones
{
    private readonly List<CombinacionVerboCodigo> _combinaciones;

    public CatalogoCombinaciones(List<CombinacionVerboCodigo> combinaciones)
    {
        _combinaciones = combinaciones;
    }

    public IEnumerable<CombinacionVerboCodigo>
CombinacionesValidas(ValidadorCombinacionVerboCodigo validador)
    {
        return _combinaciones.Where(c => validador.Validar(c).EsValida);
    }
```

```
    public IEnumerable<CombinacionVerboCodigo>
CombinacionesInvalidas(ValidadorCombinacionVerboCodigo validador)
    {
        return _combinaciones.Where(c => !validador.Validar(c).EsValida);
    }
}
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: la clase CatalogoCombinaciones agrupa las combinaciones verbo-código del proyecto y permite filtrarlas por validez.

Errores comunes:

- No inicializar _combinaciones en el constructor: la clase lanza NullReferenceException.
- Olvidar using System.Linq;: Where no se resuelve.
- Devolver List en lugar de IEnumerable: expone la implementación interna.

Verificación: la clase CatalogoCombinaciones compila sin errores y expone un constructor y dos métodos.

PASO 2.2.10 — Añadir el using necesario en CatalogoCombinaciones

- **Acción 1:** En el panel Solution Explorer, hacer doble clic en CatalogoCombinaciones.cs.
- **Acción 2:** Añadir la línea using System.Linq; al principio del archivo si no está presente.
- **Acción 3:** Verificar que no hay subrayados rojos en Where.
- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: el método Where pertenece a System.Linq. En proyectos con ImplicitUsings desactivado, hay que añadir el using explícito.

Errores comunes:

- Añadir using System.Collections.Generic; en lugar de System.Linq: no resuelve Where.
- Añadir el using después del namespace: el compilador no lo reconoce.
- Olvidar el using y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: CatalogoCombinaciones.cs compila sin errores y no muestra subrayados rojos.

PASO 2.2.11 — Crear el proyecto de tests del punto 2.2

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta tests.
- **Acción 2:** Seleccionar Add → New Project.
- **Acción 3:** Seleccionar la plantilla xUnit Test Project, nombrar el proyecto AceriaApp.CatalogoRecursos.Tests y pulsar Create.
- **Acción 4:** Verificar que el proyecto aparece en la solución y que se ha creado la carpeta tests\AceriaApp.CatalogoRecursos.Tests.

Por qué se hace: el punto 2.2 añade un proyecto de tests específico para las clases de catálogo de recursos. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

Errores comunes:

- Crear el proyecto dentro de src en lugar de tests: rompe la estructura.
- Modificar los proyectos de tests del punto 2.1: viola la regla de no modificar archivos del punto anterior.

- Usar dotnet new mstest: el curso usa xUnit.

Verificación: el proyecto AceriaApp.CatalogoRecursos.Tests aparece en la solución y su carpeta existe en tests\AceriaApp.CatalogoRecursos.Tests.

PASO 2.2.12 — Añadir la referencia del proyecto de tests a la API monolítica

- **Acción 1:** En la terminal, escribir dotnet add tests\AceriaApp.CatalogoRecursos.Tests\AceriaApp.CatalogoRecursos.Tests.csproj reference src\AceriaApp.Monolito.Api\AceriaApp.Monolito.Api.csproj y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra Reference added successfully.
- **Acción 3:** Verificar que el archivo AceriaApp.CatalogoRecursos.Tests.csproj contiene la referencia.

Por qué se hace: la referencia permite que el proyecto de tests acceda a las clases de CatalogoRecursos del proyecto monolítico.

Errores comunes:

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo .csproj: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

Verificación: el archivo AceriaApp.CatalogoRecursos.Tests.csproj contiene un ProjectReference al proyecto monolítico.

PASO 2.2.13 — Añadir el paquete FluentAssertions al proyecto de tests

- **Acción 1:** Escribir dotnet add tests\AceriaApp.CatalogoRecursos.Tests\AceriaApp.CatalogoRecursos.Tests.csproj package FluentAssertions y pulsar Enter.
- **Acción 2:** Escribir dotnet list tests\AceriaApp.CatalogoRecursos.Tests\AceriaApp.CatalogoRecursos.Tests.csproj package y pulsar Enter.
- **Acción 3:** Verificar que la salida muestra FluentAssertions con su versión.

Por qué se hace: FluentAssertions proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

Errores comunes:

- Instalar una versión preliminar: el curso usa la última estable.
- Olvidar el paquete: Should() no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

Verificación: la salida de dotnet list package muestra FluentAssertions con una versión estable.

PASO 2.2.14 — Crear el archivo ValidadorCombinacionVerboCodigoTests.cs

- **Acción 1:** En el panel Solution Explorer, expandir tests → AceriaApp.CatalogoRecursos.Tests.
- **Acción 2:** Hacer doble clic en UnitTest1.cs y renombrarlo a ValidadorCombinacionVerboCodigoTests.cs.
- **Acción 3:** Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.CatalogoRecursos;
```

```
using FluentAssertions;
using Xunit;

namespace AceriaApp.CatalogoRecursos.Tests;

public class ValidadorCombinacionVerboCodigoTests
{
    private readonly ValidadorCombinacionVerboCodigo _validador = new();

    [Fact]
    public void Validar_GetCon200_DevuelveValida()
    {
        var combinacion = new CombinacionVerboCodigo { Verbo = "GET", Codigo = 200 };

        var resultado = _validador.Validar(combinacion);

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_PostCon201_DevuelveValida()
    {
        var combinacion = new CombinacionVerboCodigo { Verbo = "POST", Codigo = 201 };

        var resultado = _validador.Validar(combinacion);

        resultado.EsValida.Should().BeTrue();
    }

    [Fact]
    public void Validar_GetCon201_DevuelveError()
    {
        var combinacion = new CombinacionVerboCodigo { Verbo = "GET", Codigo = 201 };

        var resultado = _validador.Validar(combinacion);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no permitido"));
    }

    [Fact]
    public void Validar_VerboDesconocido_DevuelveError()
    {
        var combinacion = new CombinacionVerboCodigo { Verbo = "OPTIONS", Codigo = 200 };

        var resultado = _validador.Validar(combinacion);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Verbo no permitido"));
    }
}
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: los tests verifican que el validador acepta combinaciones correctas y rechaza combinaciones con código no permitido para el verbo y con verbo desconocido.

Errores comunes:

- Olvidar el using AceriaApp.Monolito.Api.CatalogoRecursos;: las clases no se resuelven.

- Usar Assert.True en lugar de Should().BeTrue(): el curso usa FluentAssertions.
- No cubrir el caso de verbo desconocido: el test no verifica el mensaje de error.

Verificación: el archivo ValidadorCombinacionVerboCodigoTests.cs compila y contiene cuatro métodos de test.

PASO 2.2.15 — Crear el archivo CatalogoCombinacionesTests.cs

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.CatalogoRecursos.Tests.
- **Acción 2:** Seleccionar Add → Class y nombrar la clase CatalogoCombinacionesTests.cs.
- **Acción 3:** Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.CatalogoRecursos;
using FluentAssertions;
using Xunit;

namespace AceriaApp.CatalogoRecursos.Tests;

public class CatalogoCombinacionesTests
{
    [Fact]
    public void CombinacionesValidas_FiltraCorrectamente()
    {
        var combinaciones = new List<CombinacionVerboCodigo>
        {
            new() { Verbo = "GET", Codigo = 200 },
            new() { Verbo = "POST", Codigo = 201 },
            new() { Verbo = "GET", Codigo = 201 },
            new() { Verbo = "OPTIONS", Codigo = 200 }
        };
        var catalogo = new CatalogoCombinaciones(combinaciones);
        var validador = new ValidadorCombinacionVerboCodigo();

        var validas = catalogo.CombinacionesValidas(validador).ToList();
        var invalidas = catalogo.CombinacionesInvalidas(validador).ToList();

        validas.Should().HaveCount(2);
        invalidas.Should().HaveCount(2);
    }
}
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: el test verifica que el catálogo filtra correctamente las combinaciones válidas y las inválidas.

Errores comunes:

- Olvidar using System.Collections.Generic;: List no se resuelve.
- Olvidar using System.Linq;: ToList() no se resuelve.
- No comprobar los recuentos: el test podría pasar por casualidad.

Verificación: el archivo CatalogoCombinacionesTests.cs compila y contiene un método de test.

PASO 2.2.16 — Compilar la solución completa

- **Acción 1:** Escribir dotnet build AceriaApp.sln y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra Build succeeded. y 0 Warning(s), 0 Error(s).
- **Acción 3:** Comprobar que el comando termina sin errores y que el resultado coincide con la verificación del ejercicio.

Por qué se hace: compilar la solución completa verifica que todos los proyectos, incluidos los nuevos, se resuelven correctamente.

Errores comunes:

- Tener la API en ejecución: el archivo .dll está bloqueado y la compilación falla.
- No restaurar paquetes: dotnet build restaura por defecto, pero si hay un error de red, hay que ejecutar dotnet restore primero.
- Usar una versión distinta del SDK: dotnet --version debe devolver 8.x.x.

Verificación: la salida debe mostrar Build succeeded. con 0 Error(s).

PASO 2.2.17 — Ejecutar todos los tests del punto 2.2

- **Acción 1:** Escribir dotnet test tests\AceriaApp.CatalogoRecursos.Tests\AceriaApp.CatalogoRecursos.Tests.csproj y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra Passed! - Failed: 0, Passed: 5, Skipped: 0.
- **Acción 3:** Comprobar que el comando termina sin errores y que el resultado coincide con la verificación del ejercicio.

Por qué se hace: ejecutar los tests del punto 2.2 verifica que las clases de catálogo de recursos funcionan correctamente.

Errores comunes:

- Ejecutar dotnet test sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: dotnet test compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 5, Skipped: 0, Total: 5.

PASO 2.2.18 — Ejecutar todos los tests de la solución

- **Acción 1:** Escribir dotnet test AceriaApp.sln --no-build y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra Passed! - Failed: 0, Passed: 36, Skipped: 0.
- **Acción 3:** Comprobar que el comando termina sin errores y que el resultado coincide con la verificación del ejercicio.

Por qué se hace: ejecutar todos los tests de la solución verifica que los tests de los puntos 1.1 a 1.5 y de los puntos 2.1 y 2.2 pasan conjuntamente.

Errores comunes:

- No haber compilado antes: --no-build falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 36, Skipped: 0, Total: 36.

PASO 2.2.19 — Crear el README del punto 2.2

- **Acción 1:** En la carpeta del punto 2.2-recursos-verbos-codigos, hacer clic derecho y seleccionar Add → New Item → Text File.
- **Acción 2:** Nombrar el archivo README.md.
- **Acción 3:** Escribir exactamente el siguiente contenido:

```
# Punto 2.2 — Recursos, verbos HTTP y códigos de respuesta

## Objetivo
Comprender el concepto de recurso, aplicar los verbos HTTP con su semántica correcta y seleccionar el código de respuesta adecuado. Añadir al proyecto las clases de catálogo y validación de combinaciones verbo-código.

## Estado del proyecto
- Solución `AceriaApp.sln` con ocho proyectos.
- Documento `docs/catalogo-recursos.md` con el catálogo de recursos, verbos y códigos.
- Clases `CombinacionVerboCodigo`, `ValidadorCombinacionVerboCodigo` y `CatalogoCombinaciones` en `src/AceriaApp.Monolito.Api/CatalogoRecursos/`.
- Cinco tests nuevos en `tests/AceriaApp.CatalogoRecursos.Tests`.
- Los tests acumulados suman 36 pruebas unitarias.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Ejecutar `dotnet ef database update` en `src\AceriaApp.Monolito.Api`.
4. Ejecutar `dotnet run` en `src\AceriaApp.Monolito.Api`.
5. Navegar a `https://localhost:7xxx/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/catalogo-recursos.md`.
- Carpeta `CatalogoRecursos` en el proyecto monolítico con tres clases nuevas.
- Proyecto `tests/AceriaApp.CatalogoRecursos.Tests` con cinco tests nuevos.
- No se ha modificado ningún archivo del punto 2.1.

## Documentación
- [ADR-001](docs/ADR-001-monolito-vs-microservicios.md) a [ADR-006](docs/ADR-006-diseno-apis-rest.md) (heredados).
- [Catálogo de recursos](docs/catalogo-recursos.md).
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que el README del punto 2.1 no se ha modificado.

Por qué se hace: el README del punto 2.2 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 2.1.

Errores comunes:

- Modificar el README del punto 2.1: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección Qué se ha añadido respecto al punto anterior: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 2.1: rompe la separación.

Verificación: el archivo README.md existe en la carpeta del punto 2.2 con las cinco secciones.

PASO 2.2.20 — Crear el ADR-007 del punto 2.2

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta docs.

- **Acción 2:** Seleccionar Add → New Item → Text File y nombrar el archivo ADR-007-catalogo-recursos.md.
- **Acción 3:** Escribir exactamente el siguiente contenido:

```
# ADR-007: Catálogo de recursos y validación de combinaciones verbo-código

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado el estilo REST en el punto 2.1. Es necesario definir el catálogo de recursos del dominio, los verbos permitidos sobre cada recurso y los códigos de respuesta esperados, y validar automáticamente la coherencia de las combinaciones.

## Decisión
Se adopta un catálogo de recursos con cinco recursos del dominio (Orden, LineaOrden, Material, Cliente, Movimiento) y sus combinaciones verbo-URI. Se definen los códigos de respuesta esperados por verbo. Se implementa un validador automático que comprueba la coherencia de cada combinación verbo-código.

## Alternativas consideradas
- Validación manual de las combinaciones: descartado por falta de trazabilidad.
- Uso de códigos no estándar: descartado por romper la uniformidad de la API.

## Consecuencias
- Positivas: catálogo trazable, validación automática, coherencia de la API.
- Negativas: la tabla de códigos por verbo puede requerir ajuste.
- Mitigación: revisar la tabla al final del módulo 2.
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que los ADR-001 a ADR-006 no se han modificado.

Por qué se hace: el ADR-007 documenta la decisión de adoptar el catálogo de recursos y la validación automática. Es un artefacto nuevo del punto 2.2 que no modifica los ADR existentes.

Errores comunes:

- Modificar los ADR anteriores: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección Alternativas consideradas: el ADR pierde valor.
- Nombrar el archivo sin el prefijo ADR-007: rompe la convención.

Verificación: el archivo docs\ADR-007-catalogo-recursos.md existe con las cinco secciones.

PASO 2.2.21 — Verificar que no se ha modificado ningún archivo del punto 2.1

- **Acción 1:** En la terminal, escribir cd C:\cursos\AceriaApp y pulsar Enter.
- **Acción 2:** Escribir fc /L modulo-2-apis-rest-aspnet-core\2.1-diseno-apis-rest\docs\ADR-006-diseno-apis-rest.md modulo-2-apis-rest-aspnet-core\2.2-recursos-verbos-codigos\docs\ADR-006-diseno-apis-rest.md y pulsar Enter.
- **Acción 3:** Verificar que la salida muestra FC: no differences encountered.
- **Acción 4:** Escribir fc /L modulo-2-apis-rest-aspnet-core\2.1-diseno-apis-rest\docs\diseno-apis-rest.md modulo-2-apis-rest-aspnet-core\2.2-recursos-verbos-codigos\docs\diseno-apis-rest.md y pulsar Enter.
- **Acción 5:** Verificar que la salida muestra FC: no differences encountered.
- **Acción 6:** Comparar **todos** los 11 documentos Markdown heredados en lugar de limitarse a los dos ejemplos anteriores. Desde `C:\cursos\AceriaApp`, en la consola **cmd.exe** de Windows, ejecutar:

```
for %f in (modulo-2-apis-rest-aspnet-core\2.1-diseno-apis-rest\docs\*.md) do @fc.exe /L "%f"
"modulo-2-apis-rest-aspnet-core\2.2-recursos-verbos-codigos\docs\%~nxf"
```

Verificación ampliada: aparecen 11 comparaciones iguales y no falta ningún documento. En PowerShell se debe usar `fc.exe`, no el alias `fc`.

Por qué se hace: la comparación con fc verifica que los documentos del punto 2.1 no se han modificado.

Errores comunes:

- Usar comp en lugar de fc: comp compara binarios y no muestra diferencias de texto.
- Olvidar el flag /L: fc compara en modo ASCII y muestra líneas largas sin ajuste.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

Verificación: los 11 documentos del punto 2.1 no muestran diferencias.

PASO 2.2.22 — Verificar la estructura del punto 2.2

- **Acción 1:** Escribir tree /F modulo-2-apis-rest-aspnet-core\2.2-recursos-verbos-codigos y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra AceriaApp.sln, README.md, src\AceriaApp.Monolito.Api\CatalogoRecursos con las tres clases nuevas, tests\AceriaApp.CatalogoRecursos.Tests con los dos archivos de tests y docs con el catálogo de recursos y el ADR-007.
- **Acción 3:** Verificar que las carpetas Dominio, Infraestructura, Metricas, Diseno, Organizacion, BuenasPracticas y DisenoRest del punto 2.1 permanecen intactas.
- **Acción 4:** Verificar que no existe ninguna carpeta labs ni ningún archivo metaprompt.md ni verificacion.md.

Por qué se hace: la verificación estructural confirma que el punto 2.2 añade archivos nuevos sin modificar los existentes.

Errores comunes:

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 2.1: viola la regla de no modificar.
- Omitir la carpeta CatalogoRecursos: los tests no encuentran las clases.

Verificación: la estructura contiene los archivos nuevos y conserva los del punto 2.1.

PASO 2.2.23 — Verificar el .gitignore del punto 2.2

- **Acción 1:** Verificar que el archivo .gitignore existe en la carpeta del punto 2.2 (copiado del punto 2.1).
- **Acción 2:** Abrir el archivo .gitignore con Visual Studio 2026.
- **Acción 3:** Verificar que contiene las líneas bin/, obj/, *.user, *.suo, .vs/ y TestResults/.
- **Acción 4:** Verificar que no se ha modificado respecto al punto 2.1.
- **Acción 5:** Pulsar Ctrl+S si no hay cambios y cerrar el archivo.

Por qué se hace: el .gitignore se hereda del punto 2.1 sin modificaciones. La verificación confirma que no se ha tocado.

Errores comunes:

- Modificar el .gitignore copiado del punto 2.1: viola la regla de no modificar archivos del punto anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.

- Crear el .gitignore con otro nombre: el archivo no se reconoce.

Verificación: el .gitignore existe y contiene las seis líneas sin modificaciones.

PASO 2.2.24 — Hacer el commit del punto 2.2 y verificar el estado final

- **Acción 1:** Escribir cd C:\cursos\AceriaApp y pulsar Enter.
- **Acción 2:** Escribir git add . y pulsar Enter.
- **Acción 3:** Escribir git commit -m "Punto 2.2: recursos, verbos HTTP y códigos de respuesta" y pulsar Enter.
- **Acción 4:** Escribir git log --oneline y pulsar Enter para verificar los siete commits.
- **Acción 5:** Escribir git status y pulsar Enter para verificar que el árbol está limpio.

Por qué se hace: el commit del punto 2.2 registra el estado acumulado del proyecto. Los siete commits consecutivos permiten comparar el estado del proyecto entre puntos consecutivos y entre módulos completos.

Errores comunes:

- No tener configurado user.name y user.email: git commit falla.
- Añadir la carpeta bin/ y obj/ al commit: el .gitignore lo evita.
- Hacer commit antes de git add .: no hay cambios en el índice.

Verificación: git log --oneline muestra siete commits: del punto 1.1 al 1.5, del punto 2.1 y del punto 2.2. git status muestra nothing to commit, working tree clean.

Errores comunes del ejercicio completo

Error observado	Causa	Solución guiada
xcopy no copia subcarpetas	Falta el flag /E	Ejecutar xcopy origen destino /E /I /H /Y
Dictionary no se resuelve	Falta el using System.Collections.Generic;	Añadir el using al principio del archivo
Contains no se resuelve	Falta el using System.Linq;	Añadir el using al principio del archivo
Where no se resuelve	Falta el using System.Linq;	Añadir el using al principio del archivo
NullReferenceException en CatalogoCombinaciones	No se inicializa _combinaciones	Inicializar en el constructor
KeyNotFoundException en el validador	No se comprueba si el verbo existe	Comprobar ContainsKey antes de acceder
El validador acepta códigos no permitidos	Falta la comprobación con Contains	Revisar la segunda comprobación del método
Should() no se resuelve	Falta el paquete FluentAssertions	Ejecutar dotnet add package FluentAssertions
El proyecto de tests no compila	Falta la referencia al proyecto monolítico	Ejecutar dotnet add reference
La compilación falla por el .dll bloqueado	La API está en ejecución	Detener la API con Ctrl+C

Reto resuelto paso a paso

Reto: añadir un método Resumen a CatalogoCombinaciones que devuelva el número de combinaciones válidas, el número de inválidas y el porcentaje de validez, y un test que verifique el cálculo cuando hay tres válidas y dos inválidas.

Paso 1: En CatalogoCombinaciones.cs, añadir el siguiente record al final del archivo:

```
public record ResumenCombinaciones(int Validas, int Invalidas, double PorcentajeValidez);
```

Paso 2: Explicar línea por línea:

- **Línea 1:** public record ResumenCombinaciones(→ declara un contrato inmutable.
- **Línea 2:** int Validas, → número de combinaciones válidas.
- **Línea 3:** int Invalidas, → número de combinaciones inválidas.
- **Línea 4:** double PorcentajeValidez); → porcentaje de validez.

Paso 3: En CatalogoCombinaciones.cs, añadir el siguiente método dentro de la clase:

```
public ResumenCombinaciones Resumen(ValidadorCombinacionVerboCodigo validador)
{
    var validas = _combinaciones.Count(c => validador.Validar(c).EsValida);
    var invalidas = _combinaciones.Count - validas;
    var total = _combinaciones.Count;
    var porcentaje = total == 0 ? 0 : (double)validas / total * 100;
    return new ResumenCombinaciones(validas, invalidas, porcentaje);
}
```

Paso 4: Explicar línea por línea:

- **Línea 1:** public ResumenCombinaciones Resumen(ValidadorCombinacionVerboCodigo validador) → declara el método.
- **Línea 2:** { → abre el cuerpo.
- **Línea 3:** var validas = _combinaciones.Count(c => validador.Validar(c).EsValida); → cuenta las válidas.
- **Línea 4:** var invalidas = _combinaciones.Count - validas; → calcula las inválidas por diferencia.
- **Línea 5:** var total = _combinaciones.Count; → obtiene el total.
- **Línea 6:** var porcentaje = total == 0 ? 0 : (double)validas / total * 100; → calcula el porcentaje con protección contra división por cero.
- **Línea 7:** return new ResumenCombinaciones(validas, invalidas, porcentaje); → devuelve el resumen.
- **Línea 8:** } → cierra el cuerpo.

Paso 5: Guardar el archivo con Ctrl+S.

Paso 6: En CatalogoCombinacionesTests.cs, añadir el siguiente test:

```
[Fact]
public void Resumen_ConTresValidasYDosInvalidas_CalculaPorcentaje()
{
    var combinaciones = new List<CombinacionVerboCodigo>
    {
        new() { Verbo = "GET", Codigo = 200 },
```

```
        new() { Verbo = "POST", Codigo = 201 },
        new() { Verbo = "DELETE", Codigo = 204 },
        new() { Verbo = "GET", Codigo = 201 },
        new() { Verbo = "POST", Codigo = 200 }
    };
    var catalogo = new CatalogoCombinaciones(combinaciones);
    var validador = new ValidadorCombinacionVerboCodigo();

    var resumen = catalogo.Resumen(validador);

    resumen.Validas.Should().Be(3);
    resumen.Invalidas.Should().Be(2);
    resumen.PorcentajeValidez.Should().Be(60.0);
}
```

Paso 7: Explicar línea por línea:

- **Línea 1:** [Fact] → marca el método como test de xUnit.
- **Línea 2:** public void Resumen_ConTresValidasYDosInvalidas_CalculaPorcentaje() → declara el test.
- **Línea 3:** { → abre el cuerpo.
- **Línea 4:** var combinaciones = new List → crea la lista.
- **Línea 5:** { → abre el inicializador.
- **Línea 6:** new() { Verbo = "GET", Codigo = 200 }, → primera válida.
- **Línea 7:** new() { Verbo = "POST", Codigo = 201 }, → segunda válida.
- **Línea 8:** new() { Verbo = "DELETE", Codigo = 204 }, → tercera válida.
- **Línea 9:** new() { Verbo = "GET", Codigo = 201 }, → primera inválida.
- **Línea 10:** new() { Verbo = "POST", Codigo = 200 } → segunda inválida.
- **Línea 11:** }; → cierra el inicializador.
- **Línea 12:** var catalogo = new CatalogoCombinaciones(combinaciones); → instancia el catálogo.
- **Línea 13:** var validador = new ValidadorCombinacionVerboCodigo(); → instancia el validador.
- **Línea 14:** var resumen = catalogo.Resumen(validador); → invoca el método.
- **Línea 15:** resumen.Validas.Should().Be(3); → verifica las válidas.
- **Línea 16:** resumen.Invalidas.Should().Be(2); → verifica las inválidas.
- **Línea 17:** resumen.PorcentajeValidez.Should().Be(60.0); → verifica el porcentaje.
- **Línea 18:** } → cierra el cuerpo.

Paso 8: Ejecutar dotnet test tests\AceriaApp.CatalogoRecursos.Tests\AceriaApp.CatalogoRecursos.Tests.csproj y verificar que pasan los seis tests.

Paso 9: Verificar que el método Resumen no modifica ningún archivo del punto 2.1.

Paso 10: Hacer commit con git commit -m "Reto 2.2: método Resumen en CatalogoCombinaciones con cálculo de porcentaje".

Verificación: la salida de dotnet test muestra Passed: 6.

Resultado esperado

Al finalizar este punto, el alumno debe tener:

- La carpeta modulo-2-apis-rest-aspnet-core/2.2-recursos-verbos-codigos/ con la solución AceriaApp.sln que contiene nueve proyectos.
- El documento docs/catalogo-recursos.md con los recursos, verbos y códigos del proyecto.
- El documento docs/ADR-007-catalogo-recursos.md con la decisión sobre el catálogo y la validación.

- Las clases CombinacionVerboCodigo, ValidadorCombinacionVerboCodigo y CatalogoCombinaciones en src/AceriaApp.Monolito.Api/CatalogoRecursos/.
- El proyecto tests/AceriaApp.CatalogoRecursos.Tests con cinco tests (seis tras el reto).
- Los tests de los puntos 1.1 a 1.5 y 2.1 y 2.2 pasando conjuntamente con dotnet test AceriaApp.sln, total 36 pruebas (37 tras el reto).
- Los ADR-001 a ADR-006 del punto 2.1 sin modificar.
- Siete commits en Git: del punto 1.1 al 1.5, del punto 2.1 y del punto 2.2.

Conclusión y enlace al siguiente punto

El punto 2.2 ha completado el diseño de la API REST con el catálogo de recursos, la semántica de los verbos HTTP y la relación sistemática entre verbo y código de respuesta. El proyecto AceriaApp dispone ahora de un catálogo documentado de recursos, un validador automático de combinaciones verbo-código y cinco tests nuevos. Esta base permite avanzar hacia el punto 2.3, donde se estudiará la validación de peticiones con FluentValidation, se aplicarán las reglas de validación a los recursos del catálogo y se devolverán errores en formato problem details.

Fin del punto 2.2.

Punto 2.3 — Validación de peticiones

Ejercicio: Añadir al punto 2.2 un documento de validación de peticiones en docs/, una clase de validación con FluentValidation en el proyecto monolítico, un filtro genérico de validación para Minimal API y un test unitario que verifique la validación. El punto 2.3 parte de una copia completa del punto 2.2, añade archivos y modifica únicamente Program.cs de la copia 2.3 con dos líneas nuevas para registrar los validadores; la solución independiente 2.2 permanece intacta.

Ubicación de trabajo: C:\cursos\AceriaApp\modulo-2-apis-rest-aspnet-core\

PASO 2.3.1 — Crear la carpeta del punto 2.3 a partir del punto 2.2

- **Acción 1:** Abrir la terminal integrada de Visual Studio 2026 con Ctrl+ñ (o Ctrl+`` en teclados no españoles).
- **Acción 2:** Escribir cd C:\cursos\AceriaApp\modulo-2-apis-rest-aspnet-core y pulsar Enter.
- **Acción 3:** Escribir mkdir 2.3-validacion-peticiones y pulsar Enter.
- **Acción 4:** Escribir xcopy 2.2-recursos-verbos-codigos 2.3-validacion-peticiones /E /I /H /Y y pulsar Enter.
- **Acción 5:** Escribir dir 2.3-validacion-peticiones y pulsar Enter para verificar que se ha copiado el contenido.

Por qué se hace: cada punto del curso parte del contenido del punto anterior. El punto 2.3 se construye sobre el punto 2.2 copiando su contenido y añadiendo archivos nuevos sin modificar los existentes.

Errores comunes:

- Usar copy en lugar de xcopy: copy no copia carpetas de forma recursiva.
- Olvidar el flag /E: no se copian las subcarpetas vacías.
- Olvidar el flag /I: xcopy pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag /Y: xcopy pide confirmación para sobrescribir.

Verificación: al ejecutar dir 2.3-validacion-peticiones, deben aparecer AceriaApp.sln, README.md, src, tests y docs.

PASO 2.3.2 — Abrir la solución copiada en Visual Studio 2026

- **Acción 1:** Escribir cd 2.3-validacion-peticiones y pulsar Enter.
- **Acción 2:** Escribir start AceriaApp.sln y pulsar Enter.
- **Acción 3:** Verificar que Visual Studio 2026 abre la solución y muestra los proyectos heredados del punto 2.2: AceriaApp.Monolito.Api, AceriaApp.Monolito.Tests, AceriaApp.Metricas.Tests, AceriaApp.Diseno.Tests, AceriaApp.Organizacion.Tests, AceriaApp.BuenasPracticas.Tests, AceriaApp.DisenoRest.Tests y AceriaApp.CatalogoRecursos.Tests.
- **Acción 4:** Verificar que la solución compila con Ctrl+Shift+B y que la salida muestra Build succeeded..

Por qué se hace: abrir la solución copiada permite trabajar en el punto 2.3 sobre la base del punto 2.2. La compilación verifica que la copia es correcta.

Errores comunes:

- Abrir la solución del punto 2.2 en lugar de la del punto 2.3: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener la API del punto 2.2 en ejecución: el puerto queda ocupado y la compilación falla.

Verificación: Visual Studio 2026 muestra la solución AceriaApp con los ocho proyectos heredados del punto 2.2 y la compilación es correcta.

PASO 2.3.3 — Verificar que la carpeta docs contiene los documentos del punto 2.2

- **Acción 1:** En el panel Solution Explorer, expandir la carpeta docs del punto 2.3.
- **Acción 2:** Verificar que contiene ADR-001 a ADR-007, analisis-ventajas-inconvenientes.md, principios-diseno.md, organizacion-solucion.md, buenas-practicas.md, diseno-apis-rest.md y catalogo-recursos.md.
- **Acción 3:** Verificar que ninguno de los trece documentos se ha modificado.
- **Acción 4:** Verificar que la carpeta docs no contiene ningún archivo nuevo del punto 2.3 todavía.

Por qué se hace: el punto 2.3 parte del contenido del punto 2.2, que ya incluye la carpeta docs con trece documentos. No se modifica ningún documento existente.

Errores comunes:

- Modificar alguno de los ADR o documentos existentes: viola la regla de no modificar archivos del punto anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 2.2.
- Crear la carpeta docs si ya existe: mkdir falla.

Verificación: la carpeta docs contiene los trece documentos del punto 2.2 sin modificaciones.

PASO 2.3.4 — Crear el documento de validación de peticiones en docs

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta docs.
- **Acción 2:** Seleccionar Add → New Item → Text File y nombrar el archivo validacion-peticiones.md.
- **Acción 3:** Escribir exactamente el siguiente contenido:

```
# Validación de peticiones — Punto 2.3

## Niveles de validación
```

```
1. Sintáctica.
2. Semántica.
3. De negocio.

## Validación sintáctica

Comprueba el formato y los tipos de datos. Se implementa con tipos no anulables en los `record` de
petición.

## Validación semántica

Comprueba los valores y las longitudes. Se implementa con FluentValidation.

## Validación de negocio

Comprueba las reglas del dominio. Se implementa en la capa de aplicación con acceso a la base de
datos.

## Códigos de respuesta

| Nivel fallido | Código |
|---------------|--------|
| Sintáctica | 400 Bad Request |
| Semántica | 400 Bad Request |
| Negocio | 422 Unprocessable Entity |

## Formato de error

Problem details con campo `errors`.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Sintáctica | Records con tipos no anulables |
| Semántica | FluentValidation |
| Negocio | Capa de aplicación |
| Integración | Filtro genérico |
| Respuesta | Problem details 400 o 422 |
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que el archivo aparece en la carpeta docs sin subrayados rojos.

Por qué se hace: el documento de validación de peticiones es el artefacto de docs específico del punto 2.3. No modifica los trece documentos existentes del punto 2.2.

Errores comunes:

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
- Guardar el archivo en la raíz del punto en lugar de docs: rompe la estructura del repositorio.

Verificación: el archivo docs\validacion-peticiones.md existe con las siete secciones.

PASO 2.3.5 — Añadir el paquete FluentValidation.DependencyInjectionExtensions al proyecto monolítico

- **Acción 1:** En la terminal, escribir dotnet add src\AceriaApp.Monolito.Api\AceriaApp.Monolito.Api.csproj package FluentValidation.DependencyInjectionExtensions y pulsar Enter.
- **Acción 2:** Escribir dotnet list src\AceriaApp.Monolito.Api\AceriaApp.Monolito.Api.csproj package y pulsar Enter.
- **Acción 3:** Verificar que la salida muestra FluentValidation.DependencyInjectionExtensions con su versión.

Por qué se hace: FluentValidation.DependencyInjectionExtensions proporciona la integración de FluentValidation con el contenedor de dependencias de ASP.NET Core. Es necesario para registrar los validadores y para inyectarlos en los endpoints.

Errores comunes:

- Instalar FluentValidation sin AspNetCore: falta la integración con el contenedor.
- Instalar una versión preliminar: el curso usa la última estable.
- Añadir el paquete al proyecto de tests: los validadores no se resuelven en producción.

Verificación: la salida de dotnet list package muestra FluentValidation.DependencyInjectionExtensions con una versión estable.

PASO 2.3.6 — Crear la carpeta Validacion en el proyecto monolítico

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.Monolito.Api.
- **Acción 2:** Seleccionar Add → New Folder y nombrar la carpeta Validacion.
- **Acción 3:** Verificar que la carpeta Validacion aparece dentro de AceriaApp.Monolito.Api sin afectar a las carpetas existentes.
- **Acción 4:** Verificar que no se ha modificado ningún archivo existente del punto 2.2.

Por qué se hace: la carpeta Validacion albergará los validadores y el filtro genérico. Se crea como carpeta nueva sin tocar las carpetas existentes.

Errores comunes:

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: viola la regla de no modificar archivos del punto anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

Verificación: la carpeta Validacion existe dentro de AceriaApp.Monolito.Api y las carpetas existentes permanecen intactas.

PASO 2.3.7 — Crear el record CrearOrdenRequest

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta Validacion.
- **Acción 2:** Seleccionar Add → Class y nombrar la clase CrearOrdenRequest.cs.
- **Acción 3:** Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.Validacion;

public record CrearOrdenRequest(
```

```
    string Cliente,
    string Material,
    int Cantidad);
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: el record CrearOrdenRequest modela el contrato de entrada del endpoint de creación de órdenes. Los tipos no anulables aplican la validación sintáctica.

Errores comunes:

- Olvidar el namespace: la clase no se resuelve desde otros archivos.
- Usar string? en lugar de string: los campos serían opcionales y la validación sintáctica no se aplicaría.
- Usar int? en lugar de int: la cantidad sería opcional.

Verificación: el record CrearOrdenRequest compila sin errores y expone tres propiedades.

PASO 2.3.8 — Crear el validador CrearOrdenRequestValidator

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta Validacion.
- **Acción 2:** Seleccionar Add → Class y nombrar la clase CrearOrdenRequestValidator.cs.
- **Acción 3:** Escribir exactamente el siguiente código:

```
using FluentValidation;

namespace AceriaApp.Monolito.Api.Validacion;

public class CrearOrdenRequestValidator : AbstractValidator<CrearOrdenRequest>
{
    public CrearOrdenRequestValidator()
    {
        RuleFor(x => x.Cliente)
            .NotEmpty().WithMessage("El cliente es obligatorio.")
            .MaximumLength(100).WithMessage("El cliente no puede superar 100 caracteres.");

        RuleFor(x => x.Material)
            .NotEmpty().WithMessage("El material es obligatorio.")
            .MaximumLength(100).WithMessage("El material no puede superar 100 caracteres.");

        RuleFor(x => x.Cantidad)
            .GreaterThan(0).WithMessage("La cantidad debe ser mayor que cero.")
            .LessThanOrEqualTo(10000).WithMessage("La cantidad no puede superar 10000.");
    }
}
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: el validador CrearOrdenRequestValidator define las reglas semánticas del contrato de entrada. Cada regla tiene un mensaje de error específico.

Errores comunes:

- Olvidar el using FluentValidation;: AbstractValidator no se resuelve.
- Usar RuleFor con una expresión incorrecta: el validador no se ejecuta.

- No usar WithMessage: los mensajes de error son genéricos.

Verificación: el validador CrearOrdenRequestValidator compila sin errores y expone tres reglas.

PASO 2.3.9 — Crear el filtro genérico ValidationFilter

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta Validacion.
- **Acción 2:** Seleccionar Add → Class y nombrar la clase ValidationFilter.cs.
- **Acción 3:** Escribir exactamente el siguiente código:

```
using FluentValidation;

namespace AceriaApp.Monolito.Api.Validacion;

public class ValidationFilter<T> : IEndpointFilter where T : class
{
    private readonly IValidator<T> _validator;

    public ValidationFilter(IValidator<T> validator)
    {
        _validator = validator;
    }

    public async ValueTask<object?> InvokeAsync(
        EndpointFilterInvocationContext context,
        EndpointFilterDelegate next)
    {
        var request = context.Arguments.OfType<T>().FirstOrDefault();
        if (request is null)
        {
            return Results.BadRequest();
        }

        var result = await _validator.ValidateAsync(request);
        if (!result.IsValid)
        {
            var errores = result.Errors
                .GroupBy(e => e.PropertyName)
                .ToDictionary(g => g.Key, g => g.Select(e => e.ErrorMessage).ToArray());

            return Results.ValidationProblem(errores);
        }

        return await next(context);
    }
}
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: el filtro genérico ValidationFilter integra FluentValidation con Minimal API. Obtiene el validador del contenedor de dependencias, ejecuta la validación y devuelve problem details si falla.

Errores comunes:

- Olvidar el using FluentValidation;: IValidator no se resuelve.
- Olvidar el using System.Linq;: OfType, FirstOrDefault, GroupBy, ToDictionary y Select no se resuelven.
- No comprobar si la petición es nula: el filtro lanza NullReferenceException.

Verificación: el filtro ValidationFilter compila sin errores y expone un constructor y un método.

PASO 2.3.10 — Añadir los using necesarios en ValidationFilter

- **Acción 1:** En el panel Solution Explorer, hacer doble clic en ValidationFilter.cs.
- **Acción 2:** Añadir la línea using System.Linq; al principio del archivo si no está presente.
- **Acción 3:** Verificar que no hay subrayados rojos en OfType, FirstOrDefault, GroupBy, ToDictionary ni Select.
- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: los métodos OfType, FirstOrDefault, GroupBy, ToDictionary y Select pertenecen a System.Linq. En proyectos con ImplicitUsings desactivado, hay que añadir el using explícito.

Errores comunes:

- Añadir using System.Collections.Generic; en lugar de System.Linq: no resuelve los métodos.
- Añadir el using después del namespace: el compilador no lo reconoce.
- Olvidar el using y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: ValidationFilter.cs compila sin errores y no muestra subrayados rojos.

PASO 2.3.11 — Crear el proyecto de tests del punto 2.3

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta tests.
- **Acción 2:** Seleccionar Add → New Project.
- **Acción 3:** Seleccionar la plantilla xUnit Test Project, nombrar el proyecto AceriaApp.Validacion.Tests y pulsar Create.
- **Acción 4:** Verificar que el proyecto aparece en la solución y que se ha creado la carpeta tests\AceriaApp.Validacion.Tests.

Por qué se hace: el punto 2.3 añade un proyecto de tests específico para los validadores. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

Errores comunes:

- Crear el proyecto dentro de src en lugar de tests: rompe la estructura.
- Modificar los proyectos de tests del punto 2.2: viola la regla de no modificar archivos del punto anterior.
- Usar dotnet new mstest: el curso usa xUnit.

Verificación: el proyecto AceriaApp.Validacion.Tests aparece en la solución y su carpeta existe en tests\AceriaApp.Validacion.Tests.

PASO 2.3.12 — Añadir la referencia del proyecto de tests a la API monolítica

- **Acción 1:** En la terminal, escribir dotnet add tests\AceriaApp.Validacion.Tests\AceriaApp.Validacion.Tests.csproj reference src\AceriaApp.Monolito.Api\AceriaApp.Monolito.Api.csproj y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra Reference added successfully.
- **Acción 3:** Verificar que el archivo AceriaApp.Validacion.Tests.csproj contiene la referencia.

Por qué se hace: la referencia permite que el proyecto de tests acceda a los validadores del proyecto monolítico.

Errores comunes:

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo .csproj: el comando falla.

- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

Verificación: el archivo AceriaApp.Validacion.Tests.csproj contiene un ProjectReference al proyecto monolítico.

PASO 2.3.13 — Añadir el paquete FluentAssertions al proyecto de tests

- **Acción 1:** Escribir dotnet add tests\AceriaApp.Validacion.Tests\AceriaApp.Validacion.Tests.csproj package FluentAssertions y pulsar Enter.
- **Acción 2:** Escribir dotnet list tests\AceriaApp.Validacion.Tests\AceriaApp.Validacion.Tests.csproj package y pulsar Enter.
- **Acción 3:** Verificar que la salida muestra FluentAssertions con su versión.

Por qué se hace: FluentAssertions proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

Errores comunes:

- Instalar una versión preliminar: el curso usa la última estable.
- Olvidar el paquete: Should() no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

Verificación: la salida de dotnet list package muestra FluentAssertions con una versión estable.

PASO 2.3.14 — Crear el archivo CrearOrdenRequestValidatorTests.cs

- **Acción 1:** En el panel Solution Explorer, expandir tests → AceriaApp.Validacion.Tests.
- **Acción 2:** Hacer doble clic en UnitTest1.cs y renombrarlo a CrearOrdenRequestValidatorTests.cs.
- **Acción 3:** Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.Validacion;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Validacion.Tests;

public class CrearOrdenRequestValidatorTests
{
    private readonly CrearOrdenRequestValidator _validador = new();

    [Fact]
    public void Validar_PeticionCorrecta_DevuelveValida()
    {
        var peticion = new CrearOrdenRequest("ACX-001", "CHAPA-3MM", 10);

        var resultado = _validador.Validate(peticion);

        resultado.IsValid.Should().BeTrue();
    }

    [Fact]
    public void Validar_ClienteVacio_DevuelveError()
    {
        var peticion = new CrearOrdenRequest(string.Empty, "CHAPA-3MM", 10);

        var resultado = _validador.Validate(peticion);
```

```
        resultado.IsValid.Should().BeFalse();
        resultado.Errors.Should().Contain(e => e.PropertyName == "Cliente");
    }

    [Fact]
    public void Validar_CantidadNegativa_DevuelveError()
    {
        var peticion = new CrearOrdenRequest("ACX-001", "CHAPA-3MM", -5);

        var resultado = _validador.Validate(peticion);

        resultado.IsValid.Should().BeFalse();
        resultado.Errors.Should().Contain(e => e.PropertyName == "Cantidad");
    }

    [Fact]
    public void Validar_CantidadExcesiva_DevuelveError()
    {
        var peticion = new CrearOrdenRequest("ACX-001", "CHAPA-3MM", 20000);

        var resultado = _validador.Validate(peticion);

        resultado.IsValid.Should().BeFalse();
        resultado.Errors.Should().Contain(e => e.PropertyName == "Cantidad");
    }
}
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: los tests verifican que el validador acepta peticiones correctas y rechaza peticiones con cliente vacío, cantidad negativa y cantidad excesiva.

Errores comunes:

- Olvidar el using AceriaApp.Monolito.Api.Validacion;: las clases no se resuelven.
- Usar Assert.True en lugar de Should().BeTrue(): el curso usa FluentAssertions.
- No cubrir el caso de petición correcta: el test solo verifica los errores.

Verificación: el archivo CrearOrdenRequestValidatorTests.cs compila y contiene cuatro métodos de test.

PASO 2.3.15 — Crear CrearOrdenRequestTests.cs

Acción 1: En el Explorador de soluciones, abrir `tests/AceriaApp.Validacion.Tests`.

Acción 2: Agregar una clase llamada `CrearOrdenRequestTests.cs`; no crear `ValidationFilterTests.cs` porque estos dos casos verifican únicamente la asignación de propiedades del `record`.

Acción 3: Escribir este código y explicar el constructor y cada aserción:

```
using AceriaApp.Monolito.Api.Validacion;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Validacion.Tests;

public class CrearOrdenRequestTests
{
    [Fact]
```

```
    public void CrearOrdenRequest_ConDatosValidos_AsignaValores()
    {
        var peticion = new CrearOrdenRequest("ACX-001", "CHAPA-3MM", 10);

        peticion.Cliente.Should().Be("ACX-001");
        peticion.Material.Should().Be("CHAPA-3MM");
        peticion.Cantidad.Should().Be(10);
    }

    [Fact]
    public void CrearOrdenRequest_ConClienteVacio_AsignaCadenaVacia()
    {
        var peticion = new CrearOrdenRequest(string.Empty, "CHAPA-3MM", 10);

        peticion.Cliente.Should().BeEmpty();
    }
}
```

Acción 4: Guardar. Localizar el bloque `INICIO PASO 2.3.15` de la copia de referencia, descomentarlo y compilar.

Acción 5: Ejecutar los dos tests de esta clase. En el primero se comprueban `Cliente`, `Material` y `Cantidad`; en el segundo se comprueba que el record conserva la cadena vacía.

Por qué se hace: estas pruebas corresponden al contrato de datos, no al comportamiento HTTP de un filtro. El nombre de la clase debe comunicar lo que realmente se verifica.

Errores comunes: - Olvidar el `using` del espacio de nombres de producción. - Confundir creación de un record con ejecución de FluentValidation. - Conservar `ValidationFilterTests.cs` sin realizar pruebas reales del filtro.

Verificación: el archivo contiene exactamente dos métodos `[Fact]` y no existe `ValidationFilterTests.cs`.

PASO 2.3.16 — Crear ValidacionExtensions y registrar en Program.cs

Acción 1: En la carpeta `src/AceriaApp.Monolito.Api/Validacion`, crear `ValidacionExtensions.cs`.

Acción 2: Escribir y explicar el método de extensión del contenedor:

```
using FluentValidation;

namespace AceriaApp.Monolito.Api.Validacion;

public static class ValidacionExtensions
{
    public static IServiceCollection AddValidacion(this IServiceCollection services)
    {
        services.AddValidatorsFromAssemblyContaining<CrearOrdenRequestValidator>();
        return services;
    }
}
```

Acción 3: Abrir `Program.cs` de **la copia 2.3**, sin tocar el de 2.2.

Acción 4: Añadir únicamente esta línea `using`, al principio:

```
using AceriaApp.Monolito.Api.Validacion;
```

Acción 5: Después de `builder.Services.AddSwaggerGen();`, añadir esta llamada:

```
builder.Services.AddValidacion();
```

Acción 6: Activar progresivamente el bloque marcado `INICIO PASO 2.3.16` en la solución de referencia; comprobar que `Program.cs` solo incorpora esas **dos líneas aditivas**.

Acción 7: Compilar y ejecutar los tests del punto. Comprobar que el validador se registra por DI y que el código heredado sigue funcionando.

Por qué se hace: el registro del validador se concentra en una extensión nueva. El registro **no ejecuta** la validación automáticamente: para ejecutar el filtro es necesario asociarlo a un endpoint con `AddEndpointFilter`.

Errores comunes: - Registrar de nuevo el validador directamente en `Program.cs`. - No retornar `services`. - Omitir el `using` de `Validacion`. - Confundir registro en DI con la validación efectiva de cada petición.

Verificación: `ValidacionExtensions.cs` contiene `AddValidacion`, la copia 2.3 de `Program.cs` utiliza la extensión y la copia 2.2 permanece intacta.

PASO 2.3.17 — Compilar la solución completa

- **Acción 1:** Escribir dotnet build AceriaApp.sln y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra Build succeeded. y 0 Warning(s), 0 Error(s).
- **Acción 3:** Comprobar que el comando termina sin errores y que el resultado coincide con la verificación del ejercicio.

Por qué se hace: compilar la solución completa verifica que todos los proyectos, incluidos los nuevos, se resuelven correctamente.

Errores comunes:

- Tener la API en ejecución: el archivo .dll está bloqueado y la compilación falla.
- No restaurar paquetes: dotnet build restaura por defecto, pero si hay un error de red, hay que ejecutar dotnet restore primero.
- Usar una versión distinta del SDK: dotnet --version debe devolver 8.x.x.

Verificación: la salida debe mostrar Build succeeded. con 0 Error(s).

PASO 2.3.18 — Ejecutar todos los tests del punto 2.3

- **Acción 1:** Escribir dotnet test tests\AceriaApp.Validacion.Tests\AceriaApp.Validacion.Tests.csproj y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra Passed! - Failed: 0, Passed: 6, Skipped: 0.
- **Acción 3:** Comprobar que el comando termina sin errores y que el resultado coincide con la verificación del ejercicio.

Por qué se hace: ejecutar los tests del punto 2.3 verifica que el validador y el record funcionan correctamente.

Errores comunes:

- Ejecutar dotnet test sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: dotnet test compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 6, Skipped: 0, Total: 6.

PASO 2.3.19 — Ejecutar todos los tests de la solución

- **Acción 1:** Escribir dotnet test AceriaApp.sln --no-build y pulsar Enter.

- **Acción 2:** Verificar que la salida muestra Passed! - Failed: 0, Passed: 42, Skipped: 0.
- **Acción 3:** Comprobar que el comando termina sin errores y que el resultado coincide con la verificación del ejercicio.

Por qué se hace: ejecutar todos los tests de la solución verifica que los tests de los puntos anteriores y del punto 2.3 pasan conjuntamente.

Errores comunes:

- No haber compilado antes: --no-build falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 42, Skipped: 0, Total: 42.

PASO 2.3.20 — Crear el README del punto 2.3

- **Acción 1:** En la carpeta del punto 2.3-validacion-peticiones, hacer clic derecho y seleccionar Add → New Item → Text File.
- **Acción 2:** Nombrar el archivo README.md.
- **Acción 3:** Escribir exactamente el siguiente contenido:

```
# Punto 2.3 — Validación de peticiones

## Objetivo
Comprender la validación de peticiones en tres niveles y añadir al proyecto los validadores con
FluentValidation y el filtro genérico para Minimal API.

## Estado del proyecto
- Solución `AceriaApp.sln` con nueve proyectos.
- Documento `docs/validacion-peticiones.md` con los tres niveles de validación y los códigos de
respuesta.
- `record` `CrearOrdenRequest`, validador `CrearOrdenRequestValidator` y filtro
`ValidationFilter<T>` en `src/AceriaApp.Monolito.Api/Validacion/`.
- Seis tests nuevos en `tests/AceriaApp.Validacion.Tests`.
- Los tests acumulados suman 42 pruebas unitarias.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Ejecutar `dotnet ef database update` en `srcAceriaApp.Monolito.Api`.
4. Ejecutar `dotnet run` en `srcAceriaApp.Monolito.Api`.
5. Navegar a `https://localhost:7xxx/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/validacion-peticiones.md`.
- Carpeta `Validacion` en el proyecto monolítico con cuatro clases nuevas.
- Proyecto `tests/AceriaApp.Validacion.Tests` con seis tests nuevos.
- Archivo nuevo `Validacion/ValidacionExtensions.cs` con el registro de validadores.
- El `Program.cs` de la copia 2.3 incorpora solo dos líneas aditivas: `using
AceriaApp.Monolito.Api.Validacion;` y `builder.Services.AddValidacion();`.
- La solución independiente del punto 2.2 no se modifica; solo cambia `Program.cs` de la copia
2.3.

## Documentación
- [ADR-001](docs/ADR-001-monolito-vs-microservicios.md) a [ADR-007](docs/ADR-007-catalogo-
```

```
recursos.md) (heredados).
- [Validación de peticiones](docs/validacion-peticiones.md).
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que el README del punto 2.2 no se ha modificado.

Por qué se hace: el README del punto 2.3 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 2.2.

Errores comunes:

- Modificar el README del punto 2.2: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección Qué se ha añadido respecto al punto anterior: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 2.2: rompe la separación.

Verificación: el archivo README.md existe en la carpeta del punto 2.3 con las cinco secciones.

PASO 2.3.21 — Crear el ADR-008 del punto 2.3

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta docs.
- **Acción 2:** Seleccionar Add → New Item → Text File y nombrar el archivo ADR-008-validacion-peticiones.md.
- **Acción 3:** Escribir exactamente el siguiente contenido:

```
# ADR-008: Validación de peticiones con FluentValidation

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado el estilo REST y el catálogo de recursos. Es necesario definir
el contrato de entrada de cada endpoint y validarlo antes de procesar la petición.

## Decisión
Se adopta la validación en tres niveles: sintáctica con tipos no anulables, semántica con
FluentValidation y de negocio en la capa de aplicación. La integración con Minimal API se realiza
con un filtro genérico. La respuesta de validación fallida usa problem details con código 400 o
422 según el nivel.

## Alternativas consideradas
- Validación manual con condicionales: descartado por falta de uniformidad.
- Validación solo en el dominio: descartado por mezclar responsabilidades.
- DataAnnotations: descartado por menor expresividad que FluentValidation.

## Consecuencias
- Positivas: contrato de entrada explícito, mensajes de error específicos, uniformidad entre
endpoints.
- Negativas: la validación de negocio requiere acceso a datos externos.
- Mitigación: implementar la validación de negocio en la capa de aplicación.
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que los ADR-001 a ADR-007 no se han modificado.

Por qué se hace: el ADR-008 documenta la decisión de adoptar la validación en tres niveles con FluentValidation. Es un artefacto nuevo del punto 2.3 que no modifica los ADR existentes.

Errores comunes:

- Modificar los ADR anteriores: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección Alternativas consideradas: el ADR pierde valor.
- Nombrar el archivo sin el prefijo ADR-008: rompe la convención.

Verificación: el archivo docs\ADR-008-validacion-peticiones.md existe con las cinco secciones.

PASO 2.3.22 — Verificar que no se ha modificado ningún archivo del punto 2.2

- **Acción 1:** En la terminal, escribir cd C:\cursos\AceriaApp y pulsar Enter.
- **Acción 2:** Escribir fc /L modulo-2-apis-rest-aspnet-core\2.2-recursos-verbos-codigos\docs\ADR-007-catalogo-recursos.md modulo-2-apis-rest-aspnet-core\2.3-validacion-peticiones\docs\ADR-007-catalogo-recursos.md y pulsar Enter.
- **Acción 3:** Verificar que la salida muestra FC: no differences encountered.
- **Acción 4:** Escribir fc /L modulo-2-apis-rest-aspnet-core\2.2-recursos-verbos-codigos\docs\catalogo-recursos.md modulo-2-apis-rest-aspnet-core\2.3-validacion-peticiones\docs\catalogo-recursos.md y pulsar Enter.
- **Acción 5:** Verificar que la salida muestra FC: no differences encountered.
- **Acción 6:** Comparar **todos** los 13 documentos Markdown heredados en lugar de limitarse a los dos ejemplos anteriores. Desde `C:\cursos\AceriaApp`, en la consola **cmd.exe** de Windows, ejecutar:

```
for %f in (modulo-2-apis-rest-aspnet-core\2.2-recursos-verbos-codigos\docs\*.md) do @fc.exe /L
"%f" "modulo-2-apis-rest-aspnet-core\2.3-validacion-peticiones\docs\%~nxf"
```

Verificación ampliada: aparecen 13 comparaciones iguales y no falta ningún documento. En PowerShell se debe usar `fc.exe`, no el alias `fc`.

Por qué se hace: la comparación con fc verifica que los documentos del punto 2.2 no se han modificado.

Errores comunes:

- Usar comp en lugar de fc: comp compara binarios y no muestra diferencias de texto.
- Olvidar el flag /L: fc compara en modo ASCII y muestra líneas largas sin ajuste.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

Verificación: los 13 documentos del punto 2.2 no muestran diferencias.

PASO 2.3.23 — Verificar la estructura del punto 2.3

- **Acción 1:** Escribir tree /F modulo-2-apis-rest-aspnet-core\2.3-validacion-peticiones y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra AceriaApp.sln, README.md, src\AceriaApp.Monolito.Api\Validacion con las cuatro clases nuevas, tests\AceriaApp.Validacion.Tests con los dos archivos de tests y docs con el documento de validación y el ADR-008.
- **Acción 3:** Verificar que las carpetas existentes del punto 2.2 permanecen intactas.
- **Acción 4:** Verificar que no existe ninguna carpeta labs ni ningún archivo metaprompt.md ni verificacion.md.

Por qué se hace: la verificación estructural confirma que el punto 2.3 añade archivos nuevos sin modificar los existentes.

Errores comunes:

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 2.2: viola la regla de no modificar.

- Omitir la carpeta Validacion: los tests no encuentran las clases.

Verificación: la estructura contiene los archivos nuevos y conserva los del punto 2.2.

PASO 2.3.24 — Hacer el commit del punto 2.3 y verificar el estado final

- **Acción 1:** Escribir cd C:\cursos\AceriaApp y pulsar Enter.
- **Acción 2:** Escribir git add . y pulsar Enter.
- **Acción 3:** Escribir git commit -m "Punto 2.3: validación de peticiones con FluentValidation" y pulsar Enter.
- **Acción 4:** Escribir git log --oneline y pulsar Enter para verificar los ocho commits.
- **Acción 5:** Escribir git status y pulsar Enter para verificar que el árbol está limpio.

Por qué se hace: el commit del punto 2.3 registra el estado acumulado del proyecto. Los ocho commits consecutivos permiten comparar el estado del proyecto entre puntos consecutivos y entre módulos completos.

Errores comunes:

- No tener configurado user.name y user.email: git commit falla.
- Añadir la carpeta bin/ y obj/ al commit: el .gitignore lo evita.
- Hacer commit antes de git add .: no hay cambios en el índice.

Verificación: git log --oneline muestra ocho commits: del punto 1.1 al 1.5, del punto 2.1 al 2.3. git status muestra nothing to commit, working tree clean.

Errores comunes del ejercicio completo

Error observado	Causa	Solución guiada
xcopy no copia subcarpetas	Falta el flag /E	Ejecutar xcopy origen destino /E /I /H /Y
AddValidatorsFromAssemblyContaining no se resuelve	Falta el using FluentValidation;	Añadir el using al principio del archivo
CrearOrdenRequestValidator no se resuelve	Falta el using AceriaApp.Monolito.Api.Validacion;	Añadir el using al principio de Program.cs
OfType, FirstOrDefault, GroupBy no se resuelven	Falta el using System.Linq;	Añadir el using al principio del archivo
AbstractValidator no se resuelve	Falta el paquete FluentValidation.DependencyInjectionExtensions	Ejecutar dotnet add package FluentValidation.DependencyInjectionExtensions
El validador no se ejecuta	No está registrado en el contenedor	Añadir AddValidatorsFromAssemblyContaining
Should() no se resuelve	Falta el paquete FluentAssertions	Ejecutar dotnet add package FluentAssertions
El proyecto de tests no compila	Falta la referencia al proyecto monolítico	Ejecutar dotnet add reference
La compilación falla por el .dll bloqueado	La API está en ejecución	Detener la API con Ctrl+C
El commit incluye bin/ y obj/	Falta el .gitignore	Verificar que el .gitignore está presente

---

Reto resuelto paso a paso

Reto: añadir un validador ActualizarOrdenRequestValidator con reglas para un record ActualizarOrdenRequest que incluya Id como Guid y Cantidad como entero, y un test que verifique que el validador rechaza un Id vacío.

Paso 1: Crear el archivo ActualizarOrdenRequest.cs en la carpeta Validacion con el siguiente contenido:

namespace AceriaApp.Monolito.Api.Validacion;

public record ActualizarOrdenRequest(
Guid Id,
string Cliente,
string Material,
int Cantidad);

Paso 2: Explicar línea por línea:

- **Línea 1:** namespace AceriaApp.Monolito.Api.Validacion; → espacio de nombres de la carpeta Validacion.
- **Línea 2:** public record ActualizarOrdenRequest(→ declara el contrato de entrada.
- **Línea 3:** Guid Id, → identificador de la orden a actualizar.
- **Línea 4:** string Cliente, → nombre del cliente.
- **Línea 5:** string Material, → material solicitado.
- **Línea 6:** int Cantidad); → cantidad solicitada.

Paso 3: Crear el archivo ActualizarOrdenRequestValidator.cs con el siguiente contenido:

using FluentValidation;

namespace AceriaApp.Monolito.Api.Validacion;

public class ActualizarOrdenRequestValidator : AbstractValidator
{
public ActualizarOrdenRequestValidator()
{
RuleFor(x => x.Id)
.NotEmpty().WithMessage("El identificador es obligatorio.");

```
    RuleFor(x => x.Cliente)
        .NotEmpty().WithMessage("El cliente es obligatorio.")
        .MaximumLength(100).WithMessage("El cliente no puede superar 100 caracteres.");

    RuleFor(x => x.Material)
        .NotEmpty().WithMessage("El material es obligatorio.")
        .MaximumLength(100).WithMessage("El material no puede superar 100 caracteres.");

    RuleFor(x => x.Cantidad)
        .GreaterThan(0).WithMessage("La cantidad debe ser mayor que cero.")
        .LessThanOrEqualTo(10000).WithMessage("La cantidad no puede superar 10000.");
}
```

}

Paso 4: Explicar línea por línea:

- **Línea 1:** using FluentValidation; → importa la biblioteca.
- **Línea 2:** namespace AceriaApp.Monolito.Api.Validacion; → espacio de nombres.
- **Línea 3:** public class ActualizarOrdenRequestValidator : AbstractValidator → declara el validador.
- **Línea 4:** { → abre el cuerpo.
- **Línea 5:** public ActualizarOrdenRequestValidator() → constructor.
- **Línea 6:** { → abre el cuerpo del constructor.

- **Línea 7:** RuleFor(x => x.Id) → regla para el campo Id.
- **Línea 8:** .NotEmpty().WithMessage("El identificador es obligatorio."); → no puede estar vacío.
- **Línea 9:** RuleFor(x => x.Cliente) → regla para el campo Cliente.
- **Línea 10:** .NotEmpty().WithMessage("El cliente es obligatorio.") → no puede estar vacío.
- **Línea 11:** .MaximumLength(100).WithMessage("El cliente no puede superar 100 caracteres."); → longitud máxima.
- **Línea 12:** RuleFor(x => x.Material) → regla para el campo Material.
- **Línea 13:** .NotEmpty().WithMessage("El material es obligatorio.") → no puede estar vacío.
- **Línea 14:** .MaximumLength(100).WithMessage("El material no puede superar 100 caracteres."); → longitud máxima.
- **Línea 15:** RuleFor(x => x.Cantidad) → regla para el campo Cantidad.
- **Línea 16:** .GreaterThan(0).WithMessage("La cantidad debe ser mayor que cero.") → mayor que cero.
- **Línea 17:** .LessThanOrEqualTo(10000).WithMessage("La cantidad no puede superar 10000."); → límite superior.
- **Línea 18:** } → cierra el constructor.
- **Línea 19:** } → cierra la clase.

Paso 5: Crear el archivo ActualizarOrdenRequestValidatorTests.cs en el proyecto de tests con el siguiente contenido:

using AceriaApp.Monolito.Api.Validacion;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Validacion.Tests;

public class ActualizarOrdenRequestValidatorTests
{
private readonly ActualizarOrdenRequestValidator _validador = new();

```
[Fact]
public void Validar_IdVacio_DevuelveError()
{
    var peticion = new ActualizarOrdenRequest(
        Guid.Empty,
        "ACX-001",
        "CHAPA-3MM",
        10);

    var resultado = _validador.Validate(peticion);

    resultado.IsValid.Should().BeFalse();
    resultado.Errors.Should().Contain(e => e.PropertyName == "Id");
}

[Fact]
public void Validar_PeticionCorrecta_DevuelveValida()
{
    var peticion = new ActualizarOrdenRequest(
        Guid.NewGuid(),
        "ACX-001",
        "CHAPA-3MM",
        10);

    var resultado = _validador.Validate(peticion);
```

```
    resultado.IsValid.Should().BeTrue();
}
```

}

Paso 6: Explicar línea por línea:

- **Línea 1:** using AceriaApp.Monolito.Api.Validacion; → importa el espacio de nombres.
- **Línea 2:** using FluentAssertions; → importa la biblioteca de aserciones.
- **Línea 3:** using Xunit; → importa el framework de tests.
- **Línea 4:** namespace AceriaApp.Validacion.Tests; → espacio de nombres del proyecto de tests.
- **Línea 5:** public class ActualizarOrdenRequestValidatorTests → declara la clase de tests.
- **Línea 6:** { → abre el cuerpo.
- **Línea 7:** private readonly ActualizarOrdenRequestValidator _validador = new(); → instancia el validador.
- **Línea 8:** [Fact] → marca el primer test.
- **Línea 9:** public void Validar_IdVacio_DevuelveError() → declara el test.
- **Línea 10:** { → abre el cuerpo.
- **Línea 11:** var peticion = new ActualizarOrdenRequest(→ instancia la petición con Guid.Empty.
- **Línea 12:** Guid.Empty, → identificador vacío.
- **Línea 13:** "ACX-001", → cliente válido.
- **Línea 14:** "CHAPA-3MM", → material válido.
- **Línea 15:** 10); → cantidad válida.
- **Línea 16:** var resultado = _validador.Validate(peticion); → ejecuta la validación.
- **Línea 17:** resultado.IsValid.Should().BeFalse(); → verifica que es inválido.
- **Línea 18:** resultado.Errors.Should().Contain(e => e.PropertyName == "Id"); → verifica el campo del error.
- **Línea 19:** } → cierra el primer test.
- **Línea 20:** [Fact] → marca el segundo test.
- **Línea 21:** public void Validar_PeticionCorrecta_DevuelveValida() → declara el segundo test.
- **Línea 22:** { → abre el cuerpo.
- **Línea 23:** var peticion = new ActualizarOrdenRequest(→ instancia la petición con Guid.NewGuid().
- **Línea 24:** Guid.NewGuid(), → identificador válido.
- **Línea 25:** "ACX-001", → cliente válido.
- **Línea 26:** "CHAPA-3MM", → material válido.
- **Línea 27:** 10); → cantidad válida.
- **Línea 28:** var resultado = _validador.Validate(peticion); → ejecuta la validación.
- **Línea 29:** resultado.IsValid.Should().BeTrue(); → verifica que es válido.
- **Línea 30:** } → cierra el segundo test.
- **Línea 31:** } → cierra la clase.

Paso 7: Ejecutar dotnet test tests\AceriaApp.Validacion.Tests\AceriaApp.Validacion.Tests.csproj y verificar que pasan los ocho tests.

Paso 8: Verificar que los archivos nuevos no modifican ningún archivo del punto 2.2.

Paso 9: Hacer commit con git commit -m "Reto 2.3: validador ActualizarOrdenRequestValidator con regla de Id".

Verificación: la salida de dotnet test muestra Passed: 8.

Resultado esperado

Al finalizar este punto, el alumno debe tener:

- La carpeta modulo-2-apis-rest-aspnet-core/2.3-validacion-peticiones/ con la solución AceriaApp.sln que contiene diez proyectos.
- El documento docs/validacion-peticiones.md con los tres niveles y los códigos de respuesta.
- El documento docs/ADR-008-validacion-peticiones.md con la decisión sobre validación.
- El record CrearOrdenRequest, el validador CrearOrdenRequestValidator y el filtro ValidationFilter en src/AceriaApp.Monolito.Api/Validacion/.
- El proyecto tests/AceriaApp.Validacion.Tests con seis tests (ocho tras el reto).
- Los tests de los puntos 1.1 a 1.5 y 2.1 a 2.3 pasando conjuntamente con dotnet test AceriaApp.sln, total 42 pruebas (44 tras el reto).
- Los ADR-001 a ADR-007 del punto 2.2 sin modificar.
- Ocho commits en Git: del punto 1.1 al 1.5 y del punto 2.1 al 2.3.
- Program.cs de la copia 2.3 modificado solo con dos líneas aditivas: using AceriaApp.Monolito.Api.Validacion; y builder.Services.AddValidacion();.

Conclusión y enlace al siguiente punto

El punto 2.3 ha completado la validación de peticiones con los tres niveles: sintáctica, semántica y de negocio. El proyecto AceriaApp dispone ahora de un contrato de entrada explícito, un validador con FluentValidation, un filtro genérico para Minimal API y seis tests nuevos. Esta base permite avanzar hacia el punto 2.4, donde se estudiará la documentación de la API con OpenAPI, se generarán los esquemas a partir de los validadores y se publicará la documentación con Swagger UI. El punto 2.4 cierra el módulo 2 y prepara el terreno para el módulo 3, donde se estudiará la arquitectura hexagonal.

Fin del punto 2.3.

Punto 2.4 — Documentación mediante OpenAPI (Swagger)

Ejercicio: Añadir al punto 2.3 un documento de documentación OpenAPI en docs/, una clase de configuración de metadatos en el proyecto monolítico y un test unitario que verifique la configuración de OpenAPI. Cerrar el módulo 2 con el resumen del estado del proyecto. El punto 2.4 copia íntegramente el estado 2.3 y añade archivos nuevos; como integración necesaria, modifica solo Program.cs de la copia 2.4 para invocar las extensiones OpenAPI. La solución 2.3 permanece intacta.

Ubicación de trabajo: C:\cursos\AceriaApp\modulo-2-apis-rest-aspnet-core\

PASO 2.4.1 — Crear la carpeta del punto 2.4 a partir del punto 2.3

- **Acción 1:** Abrir la terminal integrada de Visual Studio 2026 con Ctrl+ñ (o Ctrl+`` en teclados no españoles).
- **Acción 2:** Escribir cd C:\cursos\AceriaApp\modulo-2-apis-rest-aspnet-core y pulsar Enter.
- **Acción 3:** Escribir mkdir 2.4-documentacion-openapi y pulsar Enter.
- **Acción 4:** Escribir xcopy 2.3-validacion-peticiones 2.4-documentacion-openapi /E /I /H /Y y pulsar Enter.
- **Acción 5:** Escribir dir 2.4-documentacion-openapi y pulsar Enter para verificar que se ha copiado el contenido.

Por qué se hace: cada punto del curso parte del contenido del punto anterior. El punto 2.4 se construye sobre el punto 2.3 copiando su contenido y añadiendo archivos nuevos sin modificar los existentes.

Errores comunes:

- Usar copy en lugar de xcopy: copy no copia carpetas de forma recursiva.
- Olvidar el flag /E: no se copian las subcarpetas vacías.
- Olvidar el flag /I: xcopy pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag /Y: xcopy pide confirmación para sobrescribir.

Verificación: al ejecutar dir 2.4-documentacion-openapi, deben aparecer AceriaApp.sln, README.md, src, tests y docs.

PASO 2.4.2 — Abrir la solución copiada en Visual Studio 2026

- **Acción 1:** Escribir cd 2.4-documentacion-openapi y pulsar Enter.
- **Acción 2:** Escribir start AceriaApp.sln y pulsar Enter.
- **Acción 3:** Verificar que Visual Studio 2026 abre la solución y muestra los proyectos heredados del punto 2.3: AceriaApp.Monolito.Api, AceriaApp.Monolito.Tests, AceriaApp.Metricas.Tests, AceriaApp.Diseno.Tests, AceriaApp.Organizacion.Tests, AceriaApp.BuenasPracticas.Tests, AceriaApp.DisenoRest.Tests, AceriaApp.CatalogoRecursos.Tests y AceriaApp.Validacion.Tests.
- **Acción 4:** Verificar que la solución compila con Ctrl+Shift+B y que la salida muestra Build succeeded..

Por qué se hace: abrir la solución copiada permite trabajar en el punto 2.4 sobre la base del punto 2.3. La compilación verifica que la copia es correcta.

Errores comunes:

- Abrir la solución del punto 2.3 en lugar de la del punto 2.4: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener la API del punto 2.3 en ejecución: el puerto queda ocupado y la compilación falla.

Verificación: Visual Studio 2026 muestra la solución AceriaApp con los nueve proyectos heredados del punto 2.3 y la compilación es correcta.

PASO 2.4.3 — Verificar que la carpeta docs contiene los documentos del punto 2.3

- **Acción 1:** En el panel Solution Explorer, expandir la carpeta docs del punto 2.4.
- **Acción 2:** Verificar que contiene ADR-001 a ADR-008, analisis-ventajas-inconvenientes.md, principios-diseno.md, organizacion-solucion.md, buenas-practicas.md, diseno-apis-rest.md, catalogo-recursos.md y validacion-peticiones.md.
- **Acción 3:** Verificar que ninguno de los quince documentos se ha modificado.
- **Acción 4:** Verificar que la carpeta docs no contiene ningún archivo nuevo del punto 2.4 todavía.

Por qué se hace: el punto 2.4 parte del contenido del punto 2.3, que ya incluye la carpeta docs con quince documentos. No se modifica ningún documento existente.

Errores comunes:

- Modificar alguno de los ADR o documentos existentes: viola la regla de no modificar archivos del punto anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 2.3.
- Crear la carpeta docs si ya existe: mkdir falla.

Verificación: la carpeta docs contiene los quince documentos del punto 2.3 sin modificaciones.

PASO 2.4.4 — Crear el documento de documentación OpenAPI en docs

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta docs.
- **Acción 2:** Seleccionar Add → New Item → Text File y nombrar el archivo documentacion-openapi.md.
- **Acción 3:** Escribir exactamente el siguiente contenido:

```
# Documentación OpenAPI — Punto 2.4

## Estándar OpenAPI

Especificación para describir APIs REST. Versión 3.0.3.

## Swagger UI

Interfaz gráfica que lee el documento OpenAPI y permite explorar la API.

## Secciones del documento

- `info`: metadatos de la API.
- `servers`: URL base.
- `paths`: endpoints.
- `components`: esquemas reutilizables.
- `security`: esquemas de autenticación.
- `tags`: agrupación de endpoints.

## Anotaciones de Minimal API

- `WithName`: nombre único del endpoint.
- `WithSummary`: resumen corto.
- `WithDescription`: descripción larga.
- `WithTags`: agrupación.
- `Produces`: tipo de respuesta.
- `ProducesProblem`: tipo de error.
- `ProducesValidationProblem`: tipo de error de validación.

## Esquemas

Los esquemas se derivan de los DTOs y se referencian con `$ref`.

## Ejemplos

Los ejemplos se añaden en los esquemas de petición y de respuesta.

## Versionado

La versión de la documentación coincide con la versión del contrato en la URL.

## Herramientas que consumen OpenAPI

- Swagger UI.
- OpenAPI Generator.
- Postman.
- ReDoc.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Metadatos | `WithName`, `WithSummary`, `WithDescription`, `WithTags` |
| Esquemas | Derivados de los DTOs |
```

```
| Ejemplos | Añadidos en los esquemas |
| Versionado | v1 en URL y documentación |
| Errores | `ProducesProblem`, `ProducesValidationProblem` |

## Cierre del módulo 2

El proyecto AceriaApp dispone de:

- API Minimal con endpoints REST.
- Recursos identificados con URIs que siguen las cinco reglas.
- Verbos HTTP con semántica correcta.
- Códigos de respuesta adecuados a cada situación.
- Validación de peticiones en tres niveles.
- Documentación OpenAPI con metadatos, esquemas y ejemplos.
- Cincuenta y una pruebas unitarias acumuladas en nueve proyectos de pruebas; diez proyectos en la solución.
- Nueve puntos completados (1.1 a 1.5 y 2.1 a 2.4).
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que el archivo aparece en la carpeta docs sin subrayados rojos.

Por qué se hace: el documento de documentación OpenAPI es el artefacto de docs específico del punto 2.4. Incluye el cierre del módulo 2 con el resumen del estado del proyecto. No modifica los quince documentos existentes del punto 2.3.

Errores comunes:

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
- Guardar el archivo en la raíz del punto en lugar de docs: rompe la estructura del repositorio.

Verificación: el archivo docs\documentacion-openapi.md existe con las once secciones.

PASO 2.4.5 — Crear la carpeta Documentacion en el proyecto monolítico

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.Monolito.Api.
- **Acción 2:** Seleccionar Add → New Folder y nombrar la carpeta Documentacion.
- **Acción 3:** Verificar que la carpeta Documentacion aparece dentro de AceriaApp.Monolito.Api sin afectar a las carpetas existentes.
- **Acción 4:** Verificar que no se ha modificado ningún archivo existente del punto 2.3.

Por qué se hace: la carpeta Documentacion albergará las clases de configuración de OpenAPI. Se crea como carpeta nueva sin tocar las carpetas existentes.

Errores comunes:

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: viola la regla de no modificar archivos del punto anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

Verificación: la carpeta Documentacion existe dentro de AceriaApp.Monolito.Api y las carpetas existentes permanecen intactas.

PASO 2.4.6 — Crear la clase MetadatosApi

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta Documentacion.
- **Acción 2:** Seleccionar Add → Class y nombrar la clase MetadatosApi.cs.

- **Acción 3:** Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.Documentacion;

public class MetadatosApi
{
    public string Titulo { get; init; } = "AceriaApp.Monolito.Api";
    public string Version { get; init; } = "v1";
    public string Descripcion { get; init; } = "API del monolito de referencia.";
    public string ContactoNombre { get; init; } = "Equipo AceriaApp";
    public string ContactoEmail { get; init; } = "equipo@acerialocal";
}
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: la clase MetadatosApi modela los metadatos que se usan en la configuración de OpenAPI. Los valores por defecto se aplican al proyecto del curso.

Errores comunes:

- Olvidar el namespace: la clase no se resuelve desde otros archivos.
- No inicializar las propiedades de tipo string: el compilador emite advertencias de nulabilidad.
- Usar set en lugar de init: permite modificar la instancia después de crearla.

Verificación: la clase MetadatosApi compila sin errores y expone cinco propiedades.

PASO 2.4.7 — Crear la clase ConfiguracionOpenApi

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta Documentacion.
- **Acción 2:** Seleccionar Add → Class y nombrar la clase ConfiguracionOpenApi.cs.
- **Acción 3:** Escribir exactamente el siguiente código:

```
namespace AceriaApp.Monolito.Api.Documentacion;

public class ConfiguracionOpenApi
{
    public MetadatosApi Metadatos { get; init; } = new();
    public bool PublicarSwaggerUI { get; init; } = true;
    public string RutaDocumento { get; init; } = "/swagger/v1/swagger.json";
    public string NombreDocumento { get; init; } = "AceriaApp.Monolito.Api v1";

    public bool EsEntornoDeDesarrollo(string entorno)
    {
        return string.Equals(entorno, "Development", StringComparison.OrdinalIgnoreCase);
    }
}
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: la clase ConfiguracionOpenApi agrupa los parámetros de la configuración de OpenAPI y expone un método para comprobar el entorno. Es el punto de entrada para la configuración de la documentación.

Errores comunes:

- Olvidar el using System;: StringComparison no se resuelve.
- No inicializar Metadatos con new(): la clase lanza NullReferenceException.
- Usar == en lugar de string.Equals con StringComparison: la comparación no es robusta.

Verificación: la clase ConfiguracionOpenApi compila sin errores y expone cuatro propiedades y un método.

PASO 2.4.8 — Añadir el using necesario en ConfiguracionOpenApi

- **Acción 1:** En el panel Solution Explorer, hacer doble clic en ConfiguracionOpenApi.cs.
- **Acción 2:** Añadir la línea using System; al principio del archivo si no está presente.
- **Acción 3:** Verificar que no hay subrayados rojos en StringComparison.
- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.

Por qué se hace: el tipo StringComparison pertenece a System. En proyectos con ImplicitUsings desactivado, hay que añadir el using explícito.

Errores comunes:

- Añadir using System.Text; en lugar de System: no resuelve StringComparison.
- Añadir el using después del namespace: el compilador no lo reconoce.
- Olvidar el using y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: ConfiguracionOpenApi.cs compila sin errores y no muestra subrayados rojos.

PASO 2.4.9 — Crear DocumentacionExtensions e integrar Swagger real

Acción 1: Crear `src/AceriaApp.Monolito.Api/Documentacion/DocumentacionExtensions.cs` en la copia 2.4. No crear `ConstructorDocumentoOpenApi`: fabricar una cadena YAML no es generar el contrato real.

Acción 2: Escribir y explicar este código:

```
using Microsoft.OpenApi.Models;

namespace AceriaApp.Monolito.Api.Documentacion;

public static class DocumentacionExtensions
{
    public static IServiceCollection AddDocumentacionOpenApi(
        this IServiceCollection services, MetadatosApi metadatos)
    {
        services.AddEndpointsApiExplorer();
        services.AddSwaggerGen(options =>
        {
            options.SwaggerDoc(metadatos.Version, new OpenApiInfo
            {
                Title = metadatos.Titulo,
                Version = metadatos.Version,
                Description = metadatos.Descripcion,
                Contact = new OpenApiContact
                {
                    Name = metadatos.ContactoNombre,
                    Email = metadatos.ContactoEmail
                }
            });
        });
        return services;
    }

    public static WebApplication UseDocumentacionOpenApi(
        this WebApplication app, ConfiguracionOpenApi configuracion)
    {
```

```
        if (configuracion.EsEntornoDeDesarrollo(app.Environment.EnvironmentName))
        {
            app.UseSwagger();
            if (configuracion.PublicarSwaggerUI)
            {
                app.UseSwaggerUI(options =>
                {
                    options.SwaggerEndpoint(
                        configuracion.RutaDocumento,
                        configuracion.NombreDocumento);
                });
            }
        }
        return app;
    }
}
```

Acción 3: Comprobar que la API tiene una referencia al paquete `Swashbuckle.AspNetCore`. En la solución de referencia, descomentar el bloque `INICIO PASO 2.4.9`.

Acción 4: En el `Program.cs` de **la copia 2.4**, añadir `using AceriaApp.Monolito.Api.Documentacion;`.

Acción 5: Sustituir las dos llamadas heredadas:

```
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
```

por la única llamada real:

```
builder.Services.AddDocumentacionOpenApi(new MetadatosApi());
```

Acción 6: Sustituir el bloque heredado que publica Swagger por:

```
app.UseDocumentacionOpenApi(new ConfiguracionOpenApi());
```

Acción 7: Guardar, compilar e iniciar la aplicación en entorno `Development`.

Acción 8: Solicitar `/swagger/v1/swagger.json` y comprobar que es JSON OpenAPI con `info.title`, `info.version` y `paths["/api/v1/ordenes"]`. En `Production`, verificar que ese endpoint no se sirve.

Por qué se hace: estas extensiones registran y publican el documento **real** generado a partir de la API. No sirve crear código sin invocarlo en `Program.cs`. La solución 2.3 anterior no se modifica.

Errores comunes: - Duplicar `AddSwaggerGen` en `Program.cs`. - Crear la extensión pero no llamarla. - Omitir `using Microsoft.OpenApi.Models;`. - No activar el bloque del paso antes de compilar. - Publicar el documento OpenAPI en producción.

Verificación: un GET real al documento Swagger en `Development` devuelve estado 200 y las rutas de la API; en `Production` devuelve 404.

PASO 2.4.10 — Crear el proyecto de tests del punto 2.4

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta tests.
- **Acción 2:** Seleccionar Add → New Project.
- **Acción 3:** Seleccionar la plantilla xUnit Test Project, nombrar el proyecto AceriaApp.Documentacion.Tests y pulsar Create.
- **Acción 4:** Verificar que el proyecto aparece en la solución y que se ha creado la carpeta tests\AceriaApp.Documentacion.Tests.

Por qué se hace: el punto 2.4 añade un proyecto de tests específico para las clases de documentación. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

Errores comunes:

- Crear el proyecto dentro de src en lugar de tests: rompe la estructura.
- Modificar los proyectos de tests del punto 2.3: viola la regla de no modificar archivos del punto anterior.
- Usar dotnet new mstest: el curso usa xUnit.

Verificación: el proyecto AceriaApp.Documentacion.Tests aparece en la solución y su carpeta existe en tests\AceriaApp.Documentacion.Tests.

PASO 2.4.11 — Añadir la referencia del proyecto de tests a la API monolítica

- **Acción 1:** En la terminal, escribir dotnet add tests\AceriaApp.Documentacion.Tests\AceriaApp.Documentacion.Tests.csproj reference src\AceriaApp.Monolito.Api\AceriaApp.Monolito.Api.csproj y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra Reference added successfully.
- **Acción 3:** Verificar que el archivo AceriaApp.Documentacion.Tests.csproj contiene la referencia.

Por qué se hace: la referencia permite que el proyecto de tests acceda a las clases de Documentacion del proyecto monolítico.

Errores comunes:

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo .csproj: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

Verificación: el archivo AceriaApp.Documentacion.Tests.csproj contiene un ProjectReference al proyecto monolítico.

PASO 2.4.12 — Añadir el paquete FluentAssertions al proyecto de tests

- **Acción 1:** Escribir dotnet add tests\AceriaApp.Documentacion.Tests\AceriaApp.Documentacion.Tests.csproj package FluentAssertions y pulsar Enter.
- **Acción 2:** Escribir dotnet list tests\AceriaApp.Documentacion.Tests\AceriaApp.Documentacion.Tests.csproj package y pulsar Enter.
- **Acción 3:** Verificar que la salida muestra FluentAssertions con su versión.

Por qué se hace: FluentAssertions proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

Errores comunes:

- Instalar una versión preliminar: el curso usa la última estable.
- Olvidar el paquete: Should() no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

Verificación: la salida de dotnet list package muestra FluentAssertions con una versión estable.

PASO 2.4.13 — Crear el archivo MetadatosApiTests.cs

- **Acción 1:** En el panel Solution Explorer, expandir tests → AceriaApp.Documentacion.Tests.
- **Acción 2:** Hacer doble clic en UnitTest1.cs y renombrarlo a MetadatosApiTests.cs.

- **Acción 3:** Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.Documentacion;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Documentacion.Tests;

public class MetadatosApiTests
{
    [Fact]
    public void MetadatosApi_ConValoresPorDefecto_DevuelveValoresEsperados()
    {
        var metadatos = new MetadatosApi();

        metadatos.Titulo.Should().Be("AceriaApp.Monolito.Api");
        metadatos.Version.Should().Be("v1");
        metadatos.Descripcion.Should().Be("API del monolito de referencia.");
        metadatos.ContactoNombre.Should().Be("Equipo AceriaApp");
        metadatos.ContactoEmail.Should().Be("equipo@acerialocal");
    }

    [Fact]
    public void MetadatosApi_ConValoresPersonalizados_DevuelveValoresPersonalizados()
    {
        var metadatos = new MetadatosApi
        {
            Titulo = "AceriaApp.Ordenes.Api",
            Version = "v2",
            Descripcion = "API de órdenes de producción."
        };

        metadatos.Titulo.Should().Be("AceriaApp.Ordenes.Api");
        metadatos.Version.Should().Be("v2");
        metadatos.Descripcion.Should().Be("API de órdenes de producción.");
    }
}
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: los tests verifican que la clase MetadatosApi devuelve los valores por defecto esperados y que acepta valores personalizados.

Errores comunes:

- Olvidar el using AceriaApp.Monolito.Api.Documentacion;: la clase no se resuelve.
- Usar Assert.Equal en lugar de Should().Be(): el curso usa FluentAssertions.
- No cubrir el caso de valores personalizados: la inicialización no se verifica.

Verificación: el archivo MetadatosApiTests.cs compila y contiene dos métodos de test.

PASO 2.4.14 — Crear el archivo ConfiguracionOpenApiTests.cs

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre el proyecto AceriaApp.Documentacion.Tests.
- **Acción 2:** Seleccionar Add → Class y nombrar la clase ConfiguracionOpenApiTests.cs.
- **Acción 3:** Escribir exactamente el siguiente código:

```
using AceriaApp.Monolito.Api.Documentacion;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Documentacion.Tests;

public class ConfiguracionOpenApiTests
{
    private readonly ConfiguracionOpenApi _configuracion = new();

    [Fact]
    public void ConfiguracionOpenApi_ConValoresPorDefecto_DevuelveValoresEsperados()
    {
        _configuracion.PublicarSwaggerUI.Should().BeTrue();
        _configuracion.RutaDocumento.Should().Be("/swagger/v1/swagger.json");
        _configuracion.NombreDocumento.Should().Be("AceriaApp.Monolito.Api v1");
    }

    [Fact]
    public void EsEntornoDeDesarrollo_ConDevelopment_DevuelveTrue()
    {
        _configuracion.EsEntornoDeDesarrollo("Development").Should().BeTrue();
    }

    [Fact]
    public void EsEntornoDeDesarrollo_ConProduction_DevuelveFalse()
    {
        _configuracion.EsEntornoDeDesarrollo("Production").Should().BeFalse();
    }

    [Fact]
    public void EsEntornoDeDesarrollo_IgnoraMayusculas()
    {
        _configuracion.EsEntornoDeDesarrollo("development").Should().BeTrue();
    }
}
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que la clase compila sin errores.

Por qué se hace: los tests verifican los valores por defecto de la configuración y el comportamiento del método EsEntornoDeDesarrollo con distintos valores de entrada.

Errores comunes:

- Olvidar el using AceriaApp.Monolito.Api.Documentacion;: la clase no se resuelve.
- Usar Assert.True en lugar de Should().BeTrue(): el curso usa FluentAssertions.
- No cubrir el caso de mayúsculas: la comparación robusta no se verifica.

Verificación: el archivo ConfiguracionOpenApiTests.cs compila y contiene cuatro métodos de test.

PASO 2.4.15 — Crear DocumentacionExtensionsTests y probar el contrato HTTP

Acción 1: Añadir al proyecto `AceriaApp.Documentacion.Tests` el paquete de integración compatible con .NET 8:

```
dotnet add tests\AceriaApp.Documentacion.Tests\AceriaApp.Documentacion.Tests.csproj package
Microsoft.AspNetCore.Mvc.Testing --version 8.0.0
```

Acción 2: Crear `src/AceriaApp.Monolito.Api/Program.Public.cs` dentro del proyecto de API para que las pruebas puedan arrancar la aplicación con `WebApplicationFactory<Program>`:

```
public partial class Program { }
```

Acción 3: Crear el archivo `tests/AceriaApp.Documentacion.Tests/DocumentacionExtensionsTests.cs` y escribir estos tres tests de integración:

```
using System.Net;
using System.Text.Json;
using FluentAssertions;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Xunit;

namespace AceriaApp.Documentacion.Tests;

public class DocumentacionExtensionsTests
{
    [Fact]
    public async Task SwaggerJson_EnDevelopment_ContieneMetadatosV1()
    {
        using var factory = new WebApplicationFactory<Program>()
            .WithWebHostBuilder(b => b.UseEnvironment("Development"));
        using var client = factory.CreateClient();
        using var response = await client.GetAsync("/swagger/v1/swagger.json");

        response.StatusCode.Should().Be(HttpStatusCode.OK);
        using var json = JsonDocument.Parse(await response.Content.ReadAsStringAsync());
        var info = json.RootElement.GetProperty("info");
        info.GetProperty("title").GetString().Should().Be("AceriaApp.Monolito.Api");
        info.GetProperty("version").GetString().Should().Be("v1");
    }

    [Fact]
    public async Task SwaggerJson_EnDevelopment_DescribeLosEndpointsReales()
    {
        using var factory = new WebApplicationFactory<Program>()
            .WithWebHostBuilder(b => b.UseEnvironment("Development"));
        using var client = factory.CreateClient();
        using var response = await client.GetAsync("/swagger/v1/swagger.json");

        response.StatusCode.Should().Be(HttpStatusCode.OK);
        using var json = JsonDocument.Parse(await response.Content.ReadAsStringAsync());
        var ordenes = json.RootElement.GetProperty("paths")
            .GetProperty("/api/v1/ordenes");
        ordenes.TryGetProperty("get", out _).Should().BeTrue();
        ordenes.TryGetProperty("post", out _).Should().BeTrue();
    }

    [Fact]
    public async Task SwaggerJson_EnProduction_NoEstaPublicado()
    {
        using var factory = new WebApplicationFactory<Program>()
            .WithWebHostBuilder(b => b.UseEnvironment("Production"));
        using var client = factory.CreateClient();
        using var response = await client.GetAsync("/swagger/v1/swagger.json");

        response.StatusCode.Should().Be(HttpStatusCode.NotFound);
```

```
    }
}
```

Acción 4: Guardar los archivos y descomentar los bloques `INICIO PASO 2.4.15` en la copia de referencia.

Acción 5: Compilar y ejecutar los tres tests de integración, junto con los dos de `MetadatosApiTests` y los cuatro de `ConfiguracionOpenApiTests`.

Por qué se hace: un test que compara dos cadenas YAML no demuestra que la API sirva un documento OpenAPI. Estas pruebas arrancan la aplicación real en memoria, descargan JSON y contrastan rutas y entorno.

Errores comunes: - Olvidar `Microsoft.AspNetCore.Mvc.Testing` o `Program.Public.cs`. - No haber conectado las extensiones en `Program.cs`. - Comprobar solo valores del contenedor de DI, sin realizar una petición HTTP. - Olvidar configurar `Development` y `Production` en los tests.

Verificación: pasan **tres** pruebas HTTP reales. El proyecto tiene **nueve** tests base en total; la solución acumula **51** tests base.

PASO 2.4.16 — Compilar la solución completa

- **Acción 1:** Escribir dotnet build AceriaApp.sln y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra Build succeeded. y 0 Warning(s), 0 Error(s).
- **Acción 3:** Comprobar que el comando termina sin errores y que el resultado coincide con la verificación del ejercicio.

Por qué se hace: compilar la solución completa verifica que todos los proyectos, incluidos los nuevos, se resuelven correctamente.

Errores comunes:

- Tener la API en ejecución: el archivo .dll está bloqueado y la compilación falla.
- No restaurar paquetes: dotnet build restaura por defecto, pero si hay un error de red, hay que ejecutar dotnet restore primero.
- Usar una versión distinta del SDK: dotnet --version debe devolver 8.x.x.

Verificación: la salida debe mostrar Build succeeded. con 0 Error(s).

PASO 2.4.17 — Ejecutar todos los tests del punto 2.4

- **Acción 1:** Escribir dotnet test tests\AceriaApp.Documentacion.Tests\AceriaApp.Documentacion.Tests.csproj y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra Passed! - Failed: 0, Passed: 9, Skipped: 0.
- **Acción 3:** Comprobar que el comando termina sin errores y que el resultado coincide con la verificación del ejercicio.

Por qué se hace: ejecutar los tests del punto 2.4 verifica que las clases de documentación funcionan correctamente.

Errores comunes:

- Ejecutar dotnet test sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: dotnet test compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 9, Skipped: 0, Total: 9.

PASO 2.4.18 — Ejecutar todos los tests de la solución

- **Acción 1:** Escribir dotnet test AceriaApp.sln --no-build y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra Passed! - Failed: 0, Passed: 51, Skipped: 0.
- **Acción 3:** Comprobar que el comando termina sin errores y que el resultado coincide con la verificación del ejercicio.

Por qué se hace: ejecutar todos los tests de la solución verifica que los tests de los puntos anteriores y del punto 2.4 pasan conjuntamente.

Errores comunes:

- No haber compilado antes: --no-build falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 51, Skipped: 0, Total: 51.

PASO 2.4.19 — Crear el README del punto 2.4

- **Acción 1:** En la carpeta del punto 2.4-documentacion-openapi, hacer clic derecho y seleccionar Add → New Item → Text File.
- **Acción 2:** Nombrar el archivo README.md.
- **Acción 3:** Escribir exactamente el siguiente contenido:

```
# Punto 2.4 — Documentación mediante OpenAPI (Swagger)

## Objetivo
Comprender el estándar OpenAPI, la generación automática de documentación y las anotaciones de
metadatos en Minimal API. Añadir al proyecto las clases de configuración de OpenAPI. Cerrar el
módulo 2 con el resumen del estado del proyecto.

## Estado del proyecto
- Solución `AceriaApp.sln` con diez proyectos.
- Documento `docs/documentacion-openapi.md` con el estándar, las secciones, las anotaciones y el
cierre del módulo 2.
- Clases `MetadatosApi`, `ConfiguracionOpenApi` y `DocumentacionExtensions` en `src/
AceriaApp.Monolito.Api/Documentacion/`.
- Nueve tests nuevos en `tests/AceriaApp.Documentacion.Tests`, incluidos tres tests HTTP del
documento real.
- Los tests acumulados suman 51 pruebas unitarias.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Ejecutar `dotnet ef database update` en `src\AceriaApp.Monolito.Api`.
4. Ejecutar `dotnet run` en `src\AceriaApp.Monolito.Api`.
5. Navegar a `https://localhost:7xxx/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/documentacion-openapi.md`.
- Carpeta `Documentacion` en el proyecto monolítico con tres clases nuevas.
- Proyecto `tests/AceriaApp.Documentacion.Tests` con nueve tests nuevos.
- Cierre del módulo 2 con el resumen del estado del proyecto.
- La solución independiente 2.3 no se modifica; `Program.cs` de la copia 2.4 se adapta para
conectar las extensiones OpenAPI.
```

```
## Documentación
- [ADR-001](docs/ADR-001-monolito-vs-microservicios.md) a [ADR-008](docs/ADR-008-validacion-peticiones.md) (heredados).
- [Documentación OpenAPI](docs/documentacion-openapi.md).
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que el README del punto 2.3 no se ha modificado.

Por qué se hace: el README del punto 2.4 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 2.3.

Errores comunes:

- Modificar el README del punto 2.3: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección Qué se ha añadido respecto al punto anterior: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 2.3: rompe la separación.

Verificación: el archivo README.md existe en la carpeta del punto 2.4 con las cinco secciones.

PASO 2.4.20 — Crear el ADR-009 del punto 2.4

- **Acción 1:** En el panel Solution Explorer, hacer clic derecho sobre la carpeta docs.
- **Acción 2:** Seleccionar Add → New Item → Text File y nombrar el archivo ADR-009-documentacion-openapi.md.
- **Acción 3:** Escribir exactamente el siguiente contenido:

```
# ADR-009: Documentación OpenAPI y cierre del módulo 2

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado el estilo REST, el catálogo de recursos y la validación de peticiones. Es necesario documentar la API de forma automática y cerrar el módulo 2.

## Decisión
Se adopta el estándar OpenAPI 3.0.3. Los metadatos se añaden con anotaciones de Minimal API. Los esquemas se derivan de los DTOs. La versión de la documentación coincide con la versión del contrato en la URL. La documentación se publica con Swagger UI en el entorno de desarrollo. Se implementa la extensión de Swagger real y se comprueba el contrato generado mediante peticiones HTTP.

## Alternativas consideradas
- Documentación manual en Markdown: descartado por quedar obsoleta.
- Uso de Swagger 2.0: descartado por ser una versión antigua.
- Publicación de Swagger UI en producción: descartado por seguridad.

## Consecuencias
- Positivas: documentación viva, generación automática, integración con herramientas.
- Negativas: la configuración de metadatos puede requerir ajuste.
- Mitigación: revisar la configuración al final del módulo 2.
```

- **Acción 4:** Pulsar Ctrl+S para guardar el archivo.
- **Acción 5:** Verificar que los ADR-001 a ADR-008 no se han modificado.

Por qué se hace: el ADR-009 documenta la decisión de adoptar OpenAPI y de cerrar el módulo 2. Es un artefacto nuevo del punto 2.4 que no modifica los ADR existentes.

Errores comunes:

- Modificar los ADR anteriores: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección Alternativas consideradas: el ADR pierde valor.
- Nombrar el archivo sin el prefijo ADR-009: rompe la convención.

Verificación: el archivo docs\ADR-009-documentacion-openapi.md existe con las cinco secciones.

PASO 2.4.21 — Verificar que no se ha modificado ningún archivo del punto 2.3

- **Acción 1:** En la terminal, escribir cd C:\cursos\AceriaApp y pulsar Enter.
- **Acción 2:** Escribir fc /L modulo-2-apis-rest-aspnet-core\2.3-validacion-peticiones\docs\ADR-008-validacion-peticiones.md modulo-2-apis-rest-aspnet-core\2.4-documentacion-openapi\docs\ADR-008-validacion-peticiones.md y pulsar Enter.
- **Acción 3:** Verificar que la salida muestra FC: no differences encountered.
- **Acción 4:** Escribir fc /L modulo-2-apis-rest-aspnet-core\2.3-validacion-peticiones\docs\validacion-peticiones.md modulo-2-apis-rest-aspnet-core\2.4-documentacion-openapi\docs\validacion-peticiones.md y pulsar Enter.
- **Acción 5:** Verificar que la salida muestra FC: no differences encountered.
- **Acción 6:** Comparar **todos** los 15 documentos Markdown heredados en lugar de limitarse a los dos ejemplos anteriores. Desde `C:\cursos\AceriaApp`, en la consola **cmd.exe** de Windows, ejecutar:

```
for %f in (modulo-2-apis-rest-aspnet-core\2.3-validacion-peticiones\docs\*.md) do @fc.exe /L "%f"
"modulo-2-apis-rest-aspnet-core\2.4-documentacion-openapi\docs\%~nxf"
```

Verificación ampliada: aparecen 15 comparaciones iguales y no falta ningún documento. En PowerShell se debe usar `fc.exe`, no el alias `fc`.

Por qué se hace: la comparación con fc verifica que los documentos del punto 2.3 no se han modificado.

Errores comunes:

- Usar comp en lugar de fc: comp compara binarios y no muestra diferencias de texto.
- Olvidar el flag /L: fc compara en modo ASCII y muestra líneas largas sin ajuste.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

Verificación: los 15 documentos del punto 2.3 no muestran diferencias.

PASO 2.4.22 — Verificar la estructura del punto 2.4

- **Acción 1:** Escribir tree /F modulo-2-apis-rest-aspnet-core\2.4-documentacion-openapi y pulsar Enter.
- **Acción 2:** Verificar que la salida muestra AceriaApp.sln, README.md, src\AceriaApp.Monolito.Api\Documentacion con las tres clases nuevas, tests\AceriaApp.Documentacion.Tests con los tres archivos de tests y docs con el documento de documentación y el ADR-009.
- **Acción 3:** Verificar que las carpetas existentes del punto 2.3 permanecen intactas.
- **Acción 4:** Verificar que no existe ninguna carpeta labs ni ningún archivo metaprompt.md ni verificacion.md.

Por qué se hace: la verificación estructural confirma que el punto 2.4 añade archivos nuevos sin modificar los existentes.

Errores comunes:

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.

- Modificar las carpetas del punto 2.3: viola la regla de no modificar.
- Omitir la carpeta Documentacion: los tests no encuentran las clases.

Verificación: la estructura contiene los archivos nuevos y conserva los del punto 2.3.

PASO 2.4.23 — Verificar el .gitignore del punto 2.4

- **Acción 1:** Verificar que el archivo .gitignore existe en la carpeta del punto 2.4 (copiado del punto 2.3).
- **Acción 2:** Abrir el archivo .gitignore con Visual Studio 2026.
- **Acción 3:** Verificar que contiene las líneas bin/, obj/, *.user, *.suo, .vs/ y TestResults/.
- **Acción 4:** Verificar que no se ha modificado respecto al punto 2.3.
- **Acción 5:** Pulsar Ctrl+S si no hay cambios y cerrar el archivo.

Por qué se hace: el .gitignore se hereda del punto 2.3 sin modificaciones. La verificación confirma que no se ha tocado.

Errores comunes:

- Modificar el .gitignore copiado del punto 2.3: viola la regla de no modificar archivos del punto anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el .gitignore con otro nombre: el archivo no se reconoce.

Verificación: el .gitignore existe y contiene las seis líneas sin modificaciones.

PASO 2.4.24 — Hacer el commit del punto 2.4 y verificar el estado final del módulo 2

- **Acción 1:** Escribir cd C:\cursos\AceriaApp y pulsar Enter.
- **Acción 2:** Escribir git add . y pulsar Enter.
- **Acción 3:** Escribir git commit -m "Punto 2.4: documentación OpenAPI y cierre del módulo 2" y pulsar Enter.
- **Acción 4:** Escribir git log --oneline y pulsar Enter para verificar los nueve commits.
- **Acción 5:** Escribir git status y pulsar Enter para verificar que el árbol está limpio.

Por qué se hace: el commit del punto 2.4 cierra el módulo 2. Los nueve commits consecutivos permiten comparar el estado del proyecto entre puntos consecutivos y entre módulos completos.

Errores comunes:

- No tener configurado user.name y user.email: git commit falla.
- Añadir la carpeta bin/ y obj/ al commit: el .gitignore lo evita.
- Hacer commit antes de git add .: no hay cambios en el índice.

Verificación: git log --oneline muestra nueve commits: del punto 1.1 al 1.5 y del punto 2.1 al 2.4. git status muestra nothing to commit, working tree clean.

Errores comunes del ejercicio completo

Error observado	Causa	Solución guiada
xcopy no copia subcarpetas	Falta el flag /E	Ejecutar xcopy origen destino /E /I /H /Y
StringComparison no se resuelve	Falta el using System;	Añadir el using al principio del archivo

Error observado	Causa	Solución guiada
Should() no se resuelve	Falta el paquete FluentAssertions	Ejecutar dotnet add package FluentAssertions
El proyecto de tests no compila	Falta la referencia al proyecto monolítico	Ejecutar dotnet add reference
NullReferenceException en ConfiguracionOpenApi	No se inicializa Metadatos con new()	Inicializar con new()
El JSON Swagger no incluye la versión	No se registró SwaggerDoc con los metadatos	Revisar DocumentacionExtensions
El endpoint Swagger no aparece	Falta la integración en Program.cs	Comprobar AddDocumentacionOpenApi y UseDocumentacionOpenApi
La compilación falla por el .dll bloqueado	La API está en ejecución	Detener la API con Ctrl+C
El commit incluye bin/ y obj/	Falta el .gitignore	Verificar que el .gitignore está presente
El commit no incluye los archivos nuevos	No se ha ejecutado git add .	Ejecutar git add . antes del commit

---

Reto resuelto paso a paso

Reto: distinguir explícitamente producción de desarrollo con el método `ConfiguracionOpenApi.EsEntornoDeProduccion` y comprobar ambas opciones en una prueba.

Paso 1: Abrir `ConfiguracionOpenApi.cs` de la copia 2.4 y localizar la clase.

Paso 2: Añadir el método siguiente conservando los métodos anteriores:

```
public bool EsEntornoDeProduccion(string entorno)
{
    return string.Equals(entorno, "Production", StringComparison.OrdinalIgnoreCase);
}
```

Paso 3: Comprobar que `EsEntornoDeDesarrollo` no ha cambiado.

Paso 4: Abrir `ConfiguracionOpenApiTests.cs` y añadir:

```
[Fact]
public void EsEntornoDeProduccion_DistingueProductionDeDevelopment()
{
    _configuracion.EsEntornoDeProduccion("Production").Should().BeTrue();
    _configuracion.EsEntornoDeProduccion("Development").Should().BeFalse();
}
```

Paso 5: Guardar y descomentar los bloques correspondientes al reto en la solución de referencia.

Paso 6: Ejecutar `dotnet test tests\AceriaApp.Documentacion.Tests\AceriaApp.Documentacion.Tests.csproj`.

Paso 7: Comprobar **10 tests** en el punto 2.4 tras realizar el reto, y **52** en total si no se han incorporado acumulativamente otros retos opcionales.

Paso 8: Registrar el cambio cuando realmente se haya completado.

Por qué se hace: distingue explícitamente los dos entornos sin recurrir a una clase que fabrique documentos ficticios.

Errores comunes: no conservar el método de desarrollo, comparar con igualdad sensible a mayúsculas o no activar el nuevo test.

Verificación: Production devuelve verdadero; Development devuelve falso, y el test lo demuestra.

Resultado esperado

Al finalizar este punto, el alumno debe tener:

- La carpeta modulo-2-apis-rest-aspnet-core/2.4-documentacion-openapi/ con la solución AceriaApp.sln que contiene once proyectos.
- El documento docs/documentacion-openapi.md con el estándar, las secciones, las anotaciones y el cierre del módulo 2.
- El documento docs/ADR-009-documentacion-openapi.md con la decisión sobre OpenAPI.
- Las clases MetadatosApi, ConfiguracionOpenApi y DocumentacionExtensions en src/AceriaApp.Monolito.Api/Documentacion/.
- El proyecto tests/AceriaApp.Documentacion.Tests con nueve tests, tres de integración HTTP (diez tras el reto).
- Los tests de los puntos 1.1 a 1.5 y 2.1 a 2.4 pasando conjuntamente con dotnet test AceriaApp.sln, total 51 pruebas (52 tras el reto).
- Los ADR-001 a ADR-008 del punto 2.3 sin modificar.
- Nueve commits en Git: del punto 1.1 al 1.5 y del punto 2.1 al 2.4.
- El módulo 2 cerrado con el resumen del estado del proyecto en docs/documentacion-openapi.md.

Conclusión y enlace al siguiente punto

El punto 2.4 ha cerrado el módulo 2 con la adopción del estándar OpenAPI, la generación automática de documentación y las anotaciones de metadatos en Minimal API. El proyecto AceriaApp dispone ahora de una API REST completa con recursos, verbos, códigos de respuesta, validación en tres niveles y documentación viva con Swagger UI. El módulo 2 queda cerrado con el resumen del estado del proyecto en docs/documentacion-openapi.md. Esta base permite avanzar hacia el módulo 3, donde se estudiará la arquitectura hexagonal: capas de la aplicación, puertos y adaptadores, separación entre dominio, aplicación e infraestructura e implementación en ASP.NET Core.

Fin del punto 2.4 y del módulo 2.